Università di Genova
DIBRIS DIPARTIMENTO
DI INFORMATICA, BIOINGEGNERIA,
ROBOTICA E INGEGNERIA DEI SISTEMI

First understand, then act! From natural language to instructions to machines in a real industrial setting

SARA SAMIEIZAFARGHANDI

Master Thesis

Università di Genova, DIBRIS Via Opera Pia, 13 16145 Genova, Italy
https://www.dibris.unige.it/

Università di Genova

MSc Computer Science
Data Science and Engineering Curriculum

First understand, then act! From natural language to instructions to machines in a real industrial setting

SARA SAMIEIZAFARGHANDI

Advisor: Prof. VIVANA MASCARDI
Dr. DOMENICO MICHELE SPINGARDI

Examiner: Prof. FRANCESCA ODONE

March, 2026

DEDICATION

I dedicate this work to my family, who have supported me in every possible way despite the distance. Your encouragement, patience, and belief in me gave me the strength to continue this journey.

A special thanks to my sister, Sahar, who has always been a source of motivation and encouragement in my life.

I am also deeply grateful to my dear friends, Hussein and Monir, who stood by me throughout this journey and made this path easier with their support and kindness.

I would also like to dedicate a special thanks to my advisor, Prof. Viviana Mascardi, whose support and guidance went far beyond the role of a supervisor.

Abstract

Computer Numerical Control (CNC) machines are widely used in modern manufacturing and are conventionally programmed using formal and deterministic languages such as ISO G-code. These languages ensure precision, reproducibility, and safety, but require specialized technical expertise. In industrial practice, CNC programming is typically performed either directly in G-code or through CAM software that automatically generates controller-specific instructions. This thesis does not aim to replace established programming practices. Instead, it investigates whether a restricted and controlled natural language interface can function as an *assistive* high-level layer for generating safe and syntactically valid CNC instructions within a well-defined domain (e.g., maintenance support and rapid adjustments), while preserving full operator validation and control.

The study focuses on a cylindrical grinding environment and models a constrained subset of ISO commands corresponding to rapid positioning (G00), linear grinding (G01), and homing operations (G28). A dataset of approximately 500 natural language–G-code pairs was constructed in collaboration with Domenico Michele Spingardi to ensure technical consistency and industrial validity.

Two main paradigms were evaluated: a structured Natural Language Understanding (NLU) pipeline implemented using the Rasa framework, and Large Language Model (LLM)-based approaches, including prompt-based inference with LLaMA-3.1 70B, supervised fine-tuning of FLAN-T5 Base, and a modular structured pipeline implemented using LangChain. All approaches were assessed under identical safety constraints requiring deterministic single-line G-code generation and strict numeric validation.

Performance was measured using intent accuracy, entity extraction accuracy, exact G-code match accuracy, and joint accuracy under predefined operational constraints. The Rasa-based structured system achieved the highest joint accuracy of 98.8%, demonstrating strong reliability due to explicit intent–entity modeling and deterministic validation rules. The fine-tuned FLAN-T5 model reached a joint accuracy of 71.1%, while the structured LangChain pipeline achieved 73%. Prompt-based inference with LLaMA-3.1 70B obtained a joint accuracy of 44.5%, highlighting limitations in numeric consistency despite strong semantic recognition.

The results reveal a trade-off between generative flexibility and deterministic reliability in safety-critical contexts. While large language models provide strong semantic generalization, structured and constraint-driven architectures offer superior stability and numeric correctness. These findings suggest that future work should explore hybrid designs combining probabilistic language understanding with deterministic validation layers to support safe and controlled natural language assistive interfaces for CNC systems.

Table of Contents

Chapter 1 Introduction and Motivation **9**

1.1 Industrial Motivation 9

1.2 Problem Statement 11

1.3 Challenges 12

1.4 Contributions of This Thesis 13

Chapter 2 Background **15**

2.1 Basics of CNC Machines and G-code 15

2.2 Task-Oriented Natural Language Understanding 16

2.2.1 Rasa 18

2.3 Large Language Models for Structured Generation 19

2.3.1 Hugging Face Transformers 21

2.3.2 Groq API for LLaMA-3.1 Inference 21

2.3.3 LangChain Framework 22

Chapter 3 Related Work **23**

3.1 Systematic Literature Review Design 23

3.2 CNC Systems and G-code Programming 25

3.3 Natural Language Interfaces for Technical Systems 28

3.4 Intent Detection and Semantic Parsing for Structured Commands 31

3.5 Large Language Models for Structured Code Generation 33

3.6 Large Language Models for CNC Programming and G-code Generation . . 36
3.7 Research Gap 37

Chapter 4 Analysis and Design **40**
4.1 Dataset 40
4.1.1 Dataset Expansion with Explicit Feed-Rate Values 43
4.2 Design 44
4.2.1 Structured Intermediate Representation 44
4.2.2 Rasa-Based Approach 45
4.2.3 LLM-Based Approach 47

Chapter 5 Implementation **50**
5.1 Processing Workflow 51
5.2 Rasa-Based Pipeline Implementation 52
5.3 Feed-Rate Resolution Mechanism 53
5.3.1 Quantitative features of the developed Rasa code 53
5.4 LLM-Based Pipeline Implementation 54
5.4.1 Quantitative Features of the Developed FLAN-T5 Code 55
5.5 Structured LLM Pipeline (LangChain) 56
5.5.1 Quantitative features of the developed Langchain code 57

Chapter 6 Experiments **60**
6.1 Rasa-Based Approach 60
6.2 LLM-Based Approach 64

Chapter 7 Discussion and Conclusions **72**
7.1 Answers to the Research Questions 73
7.2 Practical and Theoretical Implications 74
7.3 Limitations 75

7.4 Future Work 76

Appendix A Appendix: Selected Code Fragments **78**

A.1 Example Rasa Outputs 78

A.2 Example Rasa Chatbot Interaction 78

A.3 Example Outputs of the Rasa-Based System 79

A.4 Prompt Template for LLaMA-3.1 70B 79

A.5 Rasa-Based Pipeline Implementation 80

Bibliography **82**

Chapter 1

Introduction and Motivation

1.1 Industrial Motivation

Computer Numerical Control (CNC) machines are widely employed in modern manufacturing environments for high-precision and repeatable machining operations. CNC systems enable the automated control of machine tools such as mills, lathes, and surface grinders through numerical instructions. These instructions are typically expressed in G-code, a low-level programming language standardized historically under ISO 6983 and extended through controller-specific dialects.

G-code provides direct control over machine movements, including linear interpolation (e.g., G01), rapid positioning (e.g., G00), and homing procedures (e.g., G28). Each command specifies exact axis coordinates, feed rates, spindle conditions, and modal states. Because CNC machines operate in direct correspondence with physical actuators, every numerical value in a G-code program has immediate physical consequences. Incorrect coordinates, feed rates outside safe ranges, or missing modal settings may result in tool damage, material waste, or unsafe operating conditions.

Despite the precision and determinism of CNC control, the process of writing G-code programs remains technically demanding. Skilled operators typically have a clear machining specification in mind, including target coordinates, cutting depth, and feed-rate constraints. They must manually encode this specification into syntactically correct and numerically precise G-code instructions. This translation requires knowledge of controller conventions, coordinate systems, feed rate selection, and safety constraints.

In the particular case of Grinding Technology S.r.l., especially within educational demonstrations and controlled experimental settings, machining operations are often conceptualized at a high level before being encoded into controller-level instructions. Such conceptual

descriptions may refer to actions like returning to a reference position or performing a finishing pass, but they are ultimately formalized into deterministic G-code commands before execution.

This thesis does not assume that CNC machines are programmed in natural language. Instead, it investigates whether a restricted and controlled natural language interface could serve as an assistive layer for mapping high-level, conceptually expressed machining intentions into formally valid G-code instructions under strict safety constraints.

In many operational contexts, machining actions are first conceptualized at a high level before being encoded into controller-level instructions. Such conceptual descriptions may refer to actions like return to home position, make a light finishing cut, or move along X slowly. However, before execution, these intentions must always be translated into formally specified G-code commands. The process of converting qualitative task descriptions into precise numerical parameters requires careful consideration of controller conventions, coordinate systems, and safety constraints.

In conventional industrial applications, CNC machines are programmed either directly in G-code or using CAM software that generates G-code automatically. This is a well-understood and efficient process. Nevertheless, in some specific situations, like educational demonstrations, rapid prototyping, or operator-assisted programming, it could be advantageous to enable operators to specify machining tasks at a conceptual level in natural language.

If high-level machining intentions must be repeatedly translated into explicit numerical instructions in situations such as training, rapid adjustment, or maintenance support, this translation process may introduce cognitive overhead and potential inconsistencies.

First, this would add cognitive overhead to the operator, who would be required to constantly alternate between high-level task planning and low-level numerical specification. Second, there would be a risk of human error, especially in situations where numerical parameters are required to meet very specific safety and operational requirements. Third, this could also impair scalability in situations where rapid task specification or frequent reconfiguration is needed.

Recent advances in Natural Language Processing (NLP) and machine learning suggest that it may be possible to automate the translation of natural language operator commands into executable machine instructions. However, unlike many text generation tasks, CNC command generation requires deterministic structural correctness, numeric precision, and compliance with safety constraints. This industrial setting therefore provides a compelling motivation for investigating whether modern NLP techniques can be adapted to safety-critical machine control domains.

Beyond the immediate programming effort required to produce G-code, the gap between human conceptual reasoning and machine-level instruction specification represents a deeper

structural mismatch. Human operators often describe machining actions at the level of simple motion intentions, for example return to the home position, move to X10 Z-2, or perform a light finishing pass at X20 Z-5. These expressions may still include qualitative descriptors (e.g., *light*, *slow*, *rough*) and can vary in wording across operators. In contrast, CNC controllers require an explicit and fully specified command, including the motion type (e.g., *G00*, *G01*, *G28*) and precise numerical parameters such as axis coordinates and feed rate. The machine does not infer intention; it executes only the provided instruction.

This mismatch becomes particularly evident in environments where rapid reconfiguration is required. Modern manufacturing increasingly emphasizes flexibility, small-batch production, and fast turnaround times. In such contexts, reducing the friction between human instruction and machine execution can improve productivity and reduce programming overhead. However, this reduction cannot compromise safety or determinism.

Furthermore, industrial systems often demand reproducibility and traceability. Any automated translation mechanism must produce consistent outputs under identical inputs. Stochastic variability or undocumented transformation steps may hinder adoption in real-world environments where reliability is paramount. Therefore, exploring natural language interfaces for CNC machines is not merely a usability problem; it is an engineering challenge that requires balancing linguistic flexibility with operational rigor.

Recent advances in artificial intelligence, particularly in natural language processing, provide an opportunity to investigate this balance. Structured NLU systems offer controlled interpretation pipelines, while large language models offer broader generalization capabilities. Evaluating these paradigms within a strictly constrained industrial context allows us to analyze their suitability for safety-critical applications.

In addition to one-shot command translation, the industrial setting also motivates a controlled clarification mechanism. Operator commands may omit mandatory parameters (e.g., feed rate) or specify them only qualitatively (e.g., “rough” or “deep cut”). In a safety-critical CNC environment, generating a fully specified command under such ambiguity requires either conservative defaults or interactive clarification. For this reason, part of this thesis investigates an interactive natural language interface, where the system can ask the operator to confirm missing numeric values within safe ranges before emitting executable G-code.

1.2 Problem Statement

The objective of this thesis is to design and evaluate a system capable of translating short natural language operator commands into valid CNC G-code commands for a surface grinding machine. Each input command is assumed to correspond to exactly one executable

G-code instruction.

The problem can be formally described as follows. Given a natural language input sentence x, the system must produce a deterministic G-code output y such that:

- the generated command corresponds to the intended machining operation;
- all required numerical parameters (e.g., axis coordinates, feed rate) are correctly extracted or inferred;
- the resulting G-code is syntactically valid for the target controller;
- the command respects predefined operational and safety constraints.

This task involves multiple interconnected components. First, the system must identify the type of machining operation (e.g., rapid positioning, linear grinding, homing). Second, it must extract relevant parameters such as axis values (X, Z) and feed rate specifications. Third, qualitative expressions such as slow, light, or rough must be mapped to predefined numerical categories. Finally, the extracted information must be transformed into a syntactically correct G-code instruction.

Unlike general-purpose code generation, this problem is constrained by strict one-line output requirements and deterministic controller behavior. Each natural language command must correspond to a single, fully specified G-code instruction that can be executed safely without further refinement.

1.3 Challenges

The natural language to G-code translation task presents several technical and conceptual challenges.

First, natural language is inherently ambiguous and variable. The same machining intention may be expressed using multiple linguistic formulations. For example, return to the home position, Move back to the home position, and Go back to the home position may all refer to a homing operation. A robust system must generalize across such variability while preserving structural correctness.

Second, operator commands may contain implicit or qualitative information. Expressions such as slow, fast, or light cut do not directly correspond to numeric values. These qualitative descriptors must be mapped to predefined numerical ranges in a consistent and deterministic manner. This mapping introduces a layer of domain-specific reasoning beyond simple text classification.

Third, numerical parameter extraction must be precise. In CNC machining, even minor deviations in coordinate values may affect tool motion. Unlike conversational AI systems where approximate correctness may be acceptable, CNC command synthesis requires exact numeric reproduction and validation.

Fourth, the output must satisfy safety and operational constraints. Axis coordinates must lie within predefined bounds. Feed rates must remain within machine capabilities. Required parameters must not be omitted. Therefore, the system includes a validation step that enforces basic structural and numerical constraints. Specifically, the implementation verifies that each generated output consists of a single supported G-code command and that all parameters fall within predefined safe ranges. More advanced controller-level safety checks are beyond the scope of this thesis.

Finally, reproducibility and determinism are necessary in a safety-critical industrial environment. Given the same input conditions, the system should always generate the same G-code output. Nevertheless, determinism is not enough. The system must generate not only deterministically correct output but also structurally and numerically sound output consistent with the intended machining task. In other words, reliability demands both stability in generation and correctness in functionality. The stochastic variability must be managed without affecting the accuracy of the generated command.

1.4 Contributions of This Thesis

The main contributions of this thesis are the following:

- The design, construction, and expert validation of a domain-specific dataset that maps short natural language operator commands to corresponding CNC G-code instructions for a surface grinding machine.
- The development of a structured NLU pipeline based on the Rasa framework, including intent classification (identifying the type of CNC operation, e.g., G00/G01/G28), entity extraction (detecting numerical parameters such as X, Z, and feed-rate values), rule-based validation (verifying that parameters satisfy predefined safety and formatting constraints), and deterministic G-code synthesis.
- The implementation of alternative approaches based on Large Language Models, including prompt-based inference, supervised fine-tuning, and hybrid pipeline architectures for structured output generation.
- A structured literature review of 30 peer-reviewed publications spanning CNC programming, natural language interfaces for industrial systems, intent–entity modeling,

and large language model-based code generation. The review was conducted to identify existing approaches to natural languagetomachine instruction translation and to analyze their limitations in safety-critical and numerically constrained domains. The main outcome of the review is the identification of a gap in the systematic evaluation of LLM-based approaches under strict structural and controller-level safety constraints, which directly motivates the experimental comparison performed in this thesis.

- A systematic experimental comparison between structured intent–entity modeling and LLM-based generation approaches under unified evaluation metrics, including intent accuracy, entity accuracy, exact G-code match, and joint accuracy.
- An analysis of the trade-offs between generative flexibility and deterministic reliability in safety-critical CNC environments.

By integrating industrial constraints with modern NLP techniques, this thesis aims to clarify whether structured NLU pipelines or large language model approaches provide more reliable performance for controller-level command synthesis in CNC machining.

Chapter 2

Background

2.1 Basics of CNC Machines and G-code

Computer Numerical Control (CNC) machines[1] are automated manufacturing systems that execute machining operations based on numerical instructions. Unlike manually operated machines, CNC systems interpret digital programs that define precise tool movements, spindle behavior, and operational parameters. The controller processes these instructions sequentially, translating numerical values into coordinated actuator motions along machine axes.

The primary low-level programming language used in CNC machining is G-code[2]. Although standardized historically under ISO 6983, G-code is implemented in slightly different dialects depending on the machine controller manufacturer. Despite dialect differences, the core structure remains consistent: each instruction typically consists of a command identifier (e.g., G00, G01, G28), axis parameters (e.g., X, Z), and optional operational parameters such as feed rate (F).

G-code operates under a modal logic structure. Certain commands activate modes that persist across subsequent lines until explicitly changed. For example, once linear interpolation mode (G01) is activated, subsequent coordinate instructions may inherit this motion mode unless another motion command is specified. This modal behavior means that the semantics of a command often depend on prior machine state.

In surface grinding machines, which are the focus of this thesis, common operations include:

[1] CNC systems are programmable machine tools that execute operations through pre-defined numerical instructions, typically encoded in G-code.

[2] `https://www.iso.org/standard/34608.html`, accessed on March 15, 2026.

- Rapid positioning (G00), used to move the tool quickly to a specified coordinate without cutting.
- Linear grinding motion (G01), which requires an explicitly defined feed rate to control material removal.
- Homing operations (G28), which return machine axes to predefined reference positions.

The execution of these commands is deterministic. Each numeric value directly influences the physical position of the machine tool. As a consequence, correctness is not only syntactic but also operational. Axis values must lie within machine limits, feed rates must respect safe operating ranges, and required parameters must not be omitted.

Unlike high-level programming languages where minor deviations may not produce catastrophic outcomes, CNC programs are tightly coupled with physical hardware. Therefore, even a single incorrect numeric parameter can lead to tool damage, reduced precision, or unsafe operation. This tight integration between digital command and physical execution makes CNC programming a particularly constrained and safety-critical domain.

Understanding this deterministic and state-dependent nature of G-code is essential for appreciating the difficulty of automatically translating natural language instructions into executable CNC commands.

An additional characteristic of CNC systems is their dependence on coordinate systems and reference frames. Machine coordinates, workpiece coordinates, and tool offsets interact to determine final tool position. Commands such as G54–G59 activate different work coordinate systems, and absolute versus incremental positioning modes (G90 vs G91) alter how numeric values are interpreted. Although this thesis focuses on single-line commands within a fixed configuration, awareness of these mechanisms highlights the contextual sensitivity of G-code interpretation.

Moreover, CNC programs are often validated through simulation prior to execution. Simulation software verifies tool paths, detects collisions, and evaluates motion feasibility. This industrial practice reflects an important principle: correctness is not assumed but verified. Any automated natural-language-to-G-code system must therefore either enforce constraints internally or integrate validation layers analogous to simulation checks.

2.2 Task-Oriented Natural Language Understanding

Task-oriented Natural Language Understanding (NLU) systems aim to map user utterances into structured representations that can be executed by downstream systems [KD18]. A widely adopted modeling framework is the intent–entity (or intent–slot) paradigm, in which

each input is classified into an intent category while relevant parameters are extracted as structured entities [GGH+18, QLC+21].

In this paradigm, each input sentence is classified into a predefined intent category that represents the action to be performed. Simultaneously, relevant parameters are extracted as entities associated with the intent. For example, in the sentence move to X 10 with slow feed, the intent may correspond to a linear motion, while entities include the axis value (X=10) and a qualitative feed rate descriptor (slow).

Intent detection is typically formulated as a sentence-level classification problem, while entity extraction is modeled as a sequence labeling task [GGH+18]. Modern systems frequently employ transformer-based encoders to generate contextual token representations, followed by classification layers for both intent and entity prediction [WWR+21, QLC+21].

One advantage of the intent–entity paradigm is structural transparency.Because the output is represented explicitly as a structured object (e.g., JSON [Bra14]), it is possible to apply validation rules before execution. Required parameters can be checked, numeric ranges can be verified, and incomplete commands can be rejected or clarified.

This structured approach is particularly suitable for domains where actions belong to a limited and well-defined set. In CNC machining, typical operations such as rapid positioning, linear interpolation, and homing naturally correspond to discrete intent categories. Numeric parameters such as coordinates and feed rate values can be modeled as entities.

However, structured NLU pipelines also have limitations. They require predefined schemas, which must be manually designed. Adding new operation types requires extending the intent taxonomy and retraining the model. Furthermore, linguistic variability may challenge generalization if training data does not adequately cover alternative phrasings.

Despite these limitations, intent–entity modeling provides deterministic processing and interpretable intermediate representations. In industrial environments where safety and reproducibility are critical, this structural clarity offers significant advantages.

From a systems perspective, intent–entity modeling can be interpreted as a structured intermediate representation between natural language and executable code. Rather than generating machine instructions directly, the system first constructs a semantic abstraction that captures the users intention and associated parameters. This abstraction serves as a stable interface for downstream deterministic synthesis.

Such intermediate representations offer engineering advantages. They decouple language interpretation from code generation, allowing independent modification of either component. If machining constraints change, synthesis rules can be updated without retraining the language model. Conversely, improvements in entity extraction do not require redesigning the G-code generation logic.

However, this decoupling introduces rigidity. The predefined intent taxonomy constrains the action space. Extending the system to new machining operations requires schema updates and additional training data. Thus, structured NLU emphasizes reliability and interpretability at the potential expense of flexibility.

2.2.1 Rasa

Rasa is an open-source conversational AI framework designed to build task-oriented chatbots. It combines (i) a Natural Language Understanding (NLU) component for intent classification and entity extraction, and (ii) dialogue management for multi-turn interactions and clarification. In this thesis, Rasa is primarily used as a structured NLU baseline; however, its dialogue capabilities are also exploited to support safe clarification when input commands are incomplete or under-specified.

Conceptually, the Rasa-based system adopted in this thesis follows the classical intent detection and slot filling paradigm widely studied in task-oriented dialogue systems [GGH+18, QLC+21]. In this paradigm, an input utterance is decomposed into: (i) an *intent*, representing the overall action category, and (ii) a set of *entities* (or slots), representing structured parameters required for execution.

This formulation is closely related to semantic parsing approaches that map natural language into structured representations suitable for programmatic execution [KD18]. In the CNC setting considered in this thesis, intent classification corresponds to identifying the type of machine operation (e.g., *G00*, *G01*, *G28*), while entity extraction corresponds to identifying numerical parameters such as axis coordinates (X, Z) and feed-rate specifications.

Modern transformer-based encoders have been shown to significantly improve joint intent detection and slot filling performance [WWR+21]. Although Rasa allows flexible pipeline configurations, including transformer-based components, the present implementation emphasizes deterministic behavior and explicit rule-based validation to satisfy industrial safety requirements.

The structured decomposition implemented in Rasa aligns with prior work where natural language commands are mapped into formal representations for robotic or machine execution [RS20, WPC19, WRK+21]. In these approaches, language understanding is treated as a structured intermediate step before execution, rather than as direct end-to-end text generation.

In addition to intent and entity modeling, Rasa enables the integration of custom Python-based actions. In this thesis, custom actions were implemented to perform feed-rate normalization, enforce numerical safety constraints, and synthesize a single validated G-code command. This deterministic post-processing stage reflects the broader principle

of separating linguistic interpretation from constraint enforcement, a design choice that contrasts with unconstrained generative LLM inference.

While structured intent–entity systems offer explicit control and reproducibility, they rely on predefined schema design and handcrafted validation rules. As linguistic variability increases, maintaining consistency between qualitative descriptors and numerical parameters requires additional rule engineering. These limitations motivate the comparison with large language model approaches, which offer broader generalization capabilities at the cost of reduced inherent structural control.

2.3 Large Language Models for Structured Generation

Large Language Models (LLMs) are deep neural network architectures based on the Transformer framework [VSP+17], trained on extensive corpora of text and code to capture broad patterns of syntax and semantics [BMR+20], enabling diverse capabilities in text generation, reasoning, and structured output generation [SSB21, WC23, RLB22, LZW+24, NZZ+23].

Most modern LLMs are based on the Transformer architecture [VSP+17] and are trained using autoregressive language modeling objectives, where each output token is predicted conditioned on preceding tokens and the input context [BMR+20].

LLMs have demonstrated strong performance across a wide range of natural language processing tasks. These include question answering and reasoning [BMR+20], summarization and structured text generation [LZW+24, NZZ+23], and code generation or constrained structured output tasks [WC23, RLB22, SSB21].

Two common strategies for applying LLMs to downstream tasks are prompt-based inference and supervised fine-tuning. In prompt-based inference, the task is specified through carefully designed instructions without updating model parameters. In supervised fine-tuning, the model parameters are adapted using domain-specific inputoutput pairs in order to improve task alignment and output consistency.

In prompt-based inference, the task is described directly in natural language instructions and the model is expected to generate task-compliant outputs without updating its parameters, relying instead on the inherent knowledge learned during pretraining [BMR+20, WWS+22]. The model generates an output that follows the specified format based on examples or explicit formatting guidance. This approach requires minimal task-specific training and allows rapid experimentation.

In supervised fine-tuning, the model is further trained on a domain-specific dataset consisting of inputoutput pairs in order to specialize its behavior for a target task [BMR+20]. Fine-

tuning aligns the models internal representations with the target task and can improve consistency and domain-specific terminology usage.

Encoder–decoder architectures such as FLAN-T5 [CHL+24] are well-suited for instruction-following tasks because they explicitly condition output generation on input sequences and are trained through instruction fine-tuning to improve task alignment. Decoder-only models such as LLaMA [TLI+23] are autoregressive transformer architectures typically applied in prompt-based inference settings, where structured output behavior is guided through carefully designed natural language prompts rather than explicit architectural constraints.

Although LLMs are powerful and flexible, they do not inherently enforce structural or numeric constraints. Output generation is probabilistic, and numeric tokens are treated similarly to textual tokens. As a result, even small deviations in numerical values may occur. In domains requiring strict precision, such as CNC machining, this lack of inherent constraint enforcement becomes a central concern.

Nevertheless, LLMs offer advantages in linguistic flexibility and generalization. They can handle diverse phrasings without requiring manually designed schemas and may capture implicit contextual relationships that structured pipelines struggle to model.

The comparison between structured intent–entity pipelines and LLM-based generation therefore represents a fundamental architectural trade-off: deterministic interpretability and constraint enforcement versus generative flexibility and broader linguistic coverage.

This background provides the conceptual foundation necessary to understand the system architectures and experimental comparisons presented in subsequent chapters.

Large language models represent a fundamentally different architectural philosophy. Instead of explicitly separating interpretation and synthesis, LLMs implicitly perform both steps within a single generative process. The model encodes semantic understanding and output formatting simultaneously within its learned parameters.

This unified generation can be advantageous in scenarios with high linguistic variability. Because LLMs are trained on diverse corpora, they can often generalize across alternative phrasings without explicit schema design. However, this implicitness also reduces transparency. The intermediate reasoning steps are not explicitly represented unless additional prompting strategies are employed.

In deterministic control domains, this architectural difference becomes critical. Structured pipelines enforce constraints before code generation, whereas LLM-based systems typically generate first and validate afterward. The ordering of interpretation and validation affects system reliability and reproducibility. Understanding this architectural contrast is central to evaluating the two paradigms in the context of CNC command synthesis.

2.3.1 Hugging Face Transformers

Supervised fine-tuning of the FLAN-T5 Base model was conducted using the Hugging Face Transformers library [WDS+20]. This framework provides standardized implementations of transformer-based architectures [VSP+17], together with training utilities, tokenization pipelines, and reproducible evaluation tools.

The CNC dataset was formatted as paired input–output sequences, where each natural language command was mapped to its corresponding G-code instruction. Fine-tuning was formulated as a sequence-to-sequence learning task, minimizing token-level cross-entropy loss between generated G-code tokens and reference command tokens. This training paradigm follows the instruction-tuning framework described in large-scale instruction-following models [CHL+24].

The Hugging Face *Trainer* API was employed to manage batching, optimization, checkpointing, and evaluation. Hyperparameters such as learning rate, number of epochs, and batch size were selected empirically to ensure stable convergence while mitigating overfitting. Fixed data splits and deterministic configurations were used to ensure reproducibility.

Using this framework enabled systematic comparison between supervised fine-tuning and prompt-based inference under identical evaluation metrics.

2.3.2 Groq API for LLaMA-3.1 Inference

Prompt-based experiments were conducted using a large decoder-only transformer model belonging to the LLaMA family [TLI+23]. Instead of hosting the model locally, inference was performed through remote API calls to optimized hardware infrastructure, allowing execution of large-scale models without requiring local high-performance GPU resources.

Prompt-based inference relies on the few-shot and instruction-following capabilities of large language models [BMR+20, WWS+22]. Structured prompts containing task instructions and explicit safety constraints were transmitted to the model. Decoding parameters such as temperature and maximum token length were explicitly controlled to reduce stochastic variability during generation.

Since autoregressive transformer models generate tokens probabilistically [VSP+17], unconstrained decoding may lead to structural or numerical inconsistencies, including malformed commands or parameter deviations. Prior research has highlighted degeneration and instability effects in open-ended generation [HBD+19]. To mitigate these issues, generation was performed under controlled decoding settings and followed by deterministic post-generation validation.

All outputs were validated to ensure: (i) exactly one G-code line was produced, (ii) the

command began with a supported instruction (G00, G01, or G28), and (iii) all numerical parameters satisfied predefined safety ranges. This validation layer is conceptually aligned with recent work on enforcing constraints in LLM-driven robotic systems [YRST24].

This deployment strategy enabled evaluation of large-scale generative inference while maintaining controlled experimental conditions.

2.3.3 LangChain Framework

A structured LLM-based pipeline was implemented using the LangChain framework. LangChain enables composition of multi-stage language model workflows and integration of intermediate validation steps.

Rather than relying on a single prompt to jointly perform intent recognition, parameter extraction, and G-code synthesis, the task was decomposed into distinct stages. This structured decomposition aligns with recent research emphasizing constrained decoding and structured output generation for large language models [SSB21, RLB22, WSM+25].

Each stage produced intermediate structured representations that were validated before being passed to subsequent processing steps. This modular design reduces error propagation and improves numerical consistency compared to unconstrained prompt-based inference.

The LangChain-based pipeline therefore represents an intermediate architectural approach between fully structured intent–entity systems and unconstrained generative LLM inference. It combines generative flexibility with explicit structural control, which is particularly important in safety-critical CNC command synthesis.

Chapter 3

Related Work

This chapter presents a systematic analysis of the literature related to the automatic translation of natural language operator commands into executable CNC machine instructions. The reviewed research spans several domains including CNC system architectures, natural language interfaces for industrial automation, structured intent–entity modeling for task-oriented dialogue, and recent applications of large language models for structured code generation.

Rather than focusing on a single discipline, the problem addressed in this thesis lies at the intersection of manufacturing automation and natural language processing. As a result, the relevant literature originates from multiple research communities. To provide a structured overview, the selected publications were grouped into thematic categories reflecting the main technological components involved in the proposed system.

3.1 Systematic Literature Review Design

A structured literature review methodology was adopted in order to provide a transparent and reproducible overview of research related to natural language interfaces, structured command interpretation, and CNC program generation. Because the problem addressed in this thesis lies at the intersection of manufacturing automation and natural language processing, the relevant literature originates from multiple research domains rather than from a single disciplinary area.

The review was guided by four research questions:

- **RQ1:** How have CNC systems and G-code programming been studied in relation to automation, monitoring, and intelligent manufacturing?

- **RQ2:** How have natural language interfaces been used to control or interact with technical, industrial, or robotic systems?
- **RQ3:** Which intent detection and semantic parsing methods have been proposed for translating natural language into structured commands?
- **RQ4:** How have large language models been applied to structured code generation and, more specifically, to CNC and G-code related tasks?

The literature search was conducted using major academic indexing platforms including Google Scholar, IEEE Xplore, ACM Digital Library, and SpringerLink. Search queries were designed to capture publications connecting natural language processing, CNC programming, machine control, and large language model based structured generation.

Two representative search queries used during the review process are reported below:

- **Q1:** ("natural language" OR "text instruction") AND ("G-code" OR CNC) AND ("large language model" OR LLM OR GPT OR NLP)
- **Q2:** ("natural language" OR "language interface" OR "text-to-code" OR "instruction-to-code") AND ("G-code" OR CNC OR "numerical control" OR "machine tool") AND ("machine learning" OR "deep learning" OR LLM OR "large language model" OR NLP)

The inclusion criteria were defined as follows: (i) the publication had to be a peer-reviewed journal article, conference paper, or book chapter; (ii) it had to be published within the time range relevant to recent developments in machine learning and large language models; and (iii) it had to address at least one of the following themes: CNC systems and G-code, natural language interfaces for technical systems, intent detection and semantic parsing, or large language models for structured generation.

The exclusion criteria were also applied in order to maintain the relevance of the review. Publications were excluded if they were outside the selected time range, were not formally published academic works, or focused on topics unrelated to structured command interpretation, CNC programming, or natural language based control of technical systems.

Candidate publications were screened in multiple stages. Initially, approximately thirty candidate publications were identified through database searches and keyword queries. Titles and abstracts were first examined to remove clearly irrelevant studies. The remaining papers were then assessed through full-text reading in order to determine their methodological relevance to CNC systems, natural language interfaces, structured command interpretation, or large language model based code generation. After this screening process, twenty-nine

publications were retained for the final literature review. Additional relevant studies were identified through backward citation tracking from key publications.

The final set of selected papers was organized into five thematic scopes reflecting the main components of the problem studied in this thesis. Table 3.1 reports the distribution of publications across these scopes.

Thematic Scope	**Number of Papers**
CNC / G-code fundamentals and CNC-oriented machine learning	4
Natural language interfaces for industrial and robotic systems	6
Intent detection and semantic parsing for structured control	5
Large language models for structured output and code generation	6
Large language models for CNC / G-code applications	8
Total	**29**

Table 3.1: Distribution of selected publications across thematic scopes.

This thematic grouping supports a progressive analysis of the literature. The first group provides the manufacturing and CNC context. The second group examines how natural language has been used to interact with robotic and industrial systems. The third group focuses on structured intent–entity interpretation methods. The fourth group reviews large language models for structured output and code generation. Finally, the fifth group addresses the most recent studies applying large language models directly to CNC and G-code related tasks.

This organization makes it possible to compare prior work systematically and to identify the specific research gap addressed in this thesis, namely the lack of controlled comparisons between structured intent–entity pipelines and large language model based approaches under strict numeric and safety constraints for industrial CNC command generation.

3.2 CNC Systems and G-code Programming

Computer Numerical Control (CNC) machines constitute the backbone of modern automated manufacturing systems. CNC technology enables the precise control of machining operations through numerical instructions that specify tool movements, feed rates, and operational

parameters. These instructions are typically expressed in the form of G-code, a standardized programming language that defines the motion and behavior of machine tools.

The G-code language historically originates from the ISO 6983 standard and has since evolved into multiple controller-specific dialects implemented by industrial machine manufacturers. G-code commands directly control the motion of the machine along predefined axes and specify parameters such as feed rate, spindle speed, and tool position. Because CNC machines interact directly with physical equipment, each numerical parameter in a G-code command has immediate mechanical consequences. Incorrect parameter values may result in machining errors, tool damage, or unsafe operating conditions.

The central role of G-code in CNC systems has motivated extensive research into methods for analyzing, optimizing, and validating CNC programs. Latif et al. [LAYK21] provide a comprehensive review of modern CNC systems and describe the evolution of manufacturing environments toward intelligent and interconnected production systems. Their work highlights how modern CNC platforms increasingly integrate data-driven monitoring, process analytics, and automated decision-support mechanisms. These developments form part of the broader transition toward Industry 4.0 manufacturing systems.

Several studies have explored the application of machine learning techniques to improve CNC machining performance. Chen et al. [CZZ+19] investigate the use of machine learning models to predict optimal machining parameters in manufacturing environments. By analyzing historical machining data, their approach enables predictive estimation of suitable cutting conditions, improving machining efficiency and reducing trial-and-error parameter tuning.

Similarly, Nguyen et al. [NPB20] propose a machine-learning-based framework for monitoring machining processes using sensor signals collected during CNC operations. Their approach focuses on detecting abnormal operating conditions and potential process failures. By analyzing sensor data in real time, the system can identify anomalies that may indicate tool wear, machining instability, or mechanical faults.

More recent research has also examined the structure and semantics of G-code programs themselves. Schmitt et al. [SME+24] investigate techniques for analyzing G-code programs using data-driven modeling approaches. Their work demonstrates that G-code instructions can be treated as structured sequences that can be interpreted and analyzed using machine learning methods. This observation opens the possibility of applying natural language processing and machine learning techniques to interpret or generate CNC commands.

Despite these advances, most existing research focuses primarily on monitoring machining processes or optimizing machining parameters rather than on improving the interaction between human operators and CNC systems. In typical industrial workflows, CNC programs are either generated through Computer-Aided Manufacturing (CAM) software or manually written by experienced operators. The programming process therefore requires specialized

knowledge of CNC commands and controller syntax.

Allowing operators to interact with CNC systems using natural language could significantly reduce the complexity of this process. However, translating human instructions into correct and executable G-code commands introduces several challenges. Natural language instructions often contain ambiguity, implicit assumptions, and qualitative descriptions that must be converted into precise numerical parameters. For example, expressions such as grind slowly or make a light pass must be translated into specific feed-rate values that satisfy machining constraints.

Table 3.2 summarizes representative research exploring artificial intelligence applications within CNC machining environments.

Paper	**Application Domain**	**Method**	**Main Contribution**
Latif et al. (2021) [LAYK21]	CNC control systems	Industrial systems review	Overview of G-code, STEP/STEP-NC, and open-architecture CNC technologies for modern manufacturing systems
Nguyen et al. (2020) [NPB20]	CNC programming	CAPP integration	Integration of computer-aided process planning with automated G-code generation using macro programming
Chen et al. (2019) [CZZ+19]	Machine tool systems	Machine learning analysis	Analysis of machine tool research and industrial development trends using machine learning techniques
Schmitt et al. (2024) [SME+24]	CNC milling	Machine learning modeling	Using machine learning to analyze G-code programs and predict energy consumption during machining

Table 3.2: Representative studies applying artificial intelligence and data-driven techniques to CNC machining environments.

Although these studies demonstrate the growing integration of artificial intelligence techniques in manufacturing environments, they do not address the problem of translating natural language operator instructions into executable CNC programs. Most existing work

focuses either on monitoring machining processes or on optimizing machining parameters using sensor data and predictive models.

Consequently, a gap remains between intelligent manufacturing systems and intuitive humanmachine interaction. Bridging this gap requires the integration of natural language understanding techniques capable of interpreting operator commands and converting them into structured control instructions. The next section therefore examines research on natural language interfaces and language-based control systems for technical and robotic applications.

3.3 Natural Language Interfaces for Technical Systems

The use of natural language as a medium for interacting with technical systems has received increasing attention in recent years. Traditional humanmachine interfaces often require operators to learn specialized command languages or graphical interfaces, which may limit accessibility and increase the complexity of system operation. Natural language interfaces aim to address this limitation by allowing users to express instructions in everyday language while the system interprets these instructions and translates them into executable actions.

Research on natural language interfaces spans several domains, including robotics, industrial control systems, and network management platforms. A central challenge in these systems is the transformation of ambiguous human language into structured representations that can be interpreted by machines. This transformation typically requires a combination of language understanding, semantic grounding, and task-oriented reasoning.

Early research explored the integration of natural language understanding techniques with domain knowledge bases. Recupero et al. [RS20] propose a knowledge-based framework for interpreting natural language instructions in network management systems. Their approach combines linguistic processing with structured knowledge representations, allowing the system to map user queries to specific network operations. The study demonstrates that incorporating domain knowledge significantly improves the interpretation of technical commands expressed in natural language.

In robotics and embodied systems, grounding natural language instructions in physical actions is an important research problem. Nyga et al. [NRP+18] investigate methods for grounding symbolic representations of language in robot actions. Their framework connects linguistic commands with structured action representations that can be executed by robotic systems. This work highlights the importance of linking language understanding with structured representations of physical actions.

More recent research has explored neural approaches to mapping natural language instructions to structured task representations. Walker et al. [WPC19] propose a neural semantic

parsing approach for translating natural language commands into formal representations of tasks. Their work demonstrates that neural models can learn to interpret complex instructions while preserving structured output formats required for task execution.

Similarly, Wang et al. [WRK+21] investigate learning-based approaches for translating natural language instructions into logical task specifications. Their framework combines neural language models with symbolic reasoning mechanisms, allowing systems to interpret instructions while maintaining structured representations of tasks. This hybrid approach illustrates the potential benefits of combining statistical language models with symbolic representations in task-oriented systems.

In industrial environments, natural language interfaces are also being explored for coordinating complex manufacturing workflows. Lim et al. [LVHK24] propose a multi-agent framework that allows natural language commands to be interpreted and distributed across multiple autonomous agents in manufacturing systems. Their system enables operators to issue high-level commands that are then decomposed into coordinated actions across multiple machines or agents.

Another important direction involves improving the robustness of intent interpretation in conversational interfaces. Saha et al. [SHP22] examine techniques for improving intent detection in conversational systems, focusing on the reliability of natural language interpretation in task-oriented dialogue environments. Their study demonstrates that robust intent detection is essential for ensuring that systems correctly interpret user instructions.

Table 3.3 summarizes representative studies on natural language interfaces for technical systems.

Paper	Application Domain	Approach	Main Contribution
Recupero et al. (2020) [RS20]	Humanoid robotics	Knowledge-based NLP	Mapping natural language expressions to humanoid robot action commands through structured parsing and domain knowledge
Nyga et al. (2018) [NRP+18]	Robotics	Symbol grounding	Linking linguistic instructions to structured robotic action representations under incomplete world knowledge
Walker et al. (2019) [WPC19]	Service robotics	Neural semantic parsing	Translating natural language commands into structured task representations for robot command understanding
Wang et al. (2021) [WRK+21]	Robotics task planning	Neural-symbolic semantic parsing	Mapping natural language instructions to executable logical task specifications (LTL)
Lim et al. (2024) [LVHK24]	Manufacturing systems	LLM-enabled multi-agent framework	Natural language coordination and control of distributed industrial agents in manufacturing environments
Saha et al. (2022) [SHP22]	Industrial network management	Intent-based NLP interface	Translating natural language instructions into executable industrial network management actions

Table 3.3: Representative research on natural language interfaces for technical and industrial systems.

Although these studies demonstrate the feasibility of interpreting natural language commands for controlling complex technical systems, most of them focus on robotics, conversational systems, or distributed industrial environments rather than on CNC machining

systems. Furthermore, many of these approaches assume structured task representations or symbolic action models that differ from the numerical precision required in CNC programming.

In CNC machining, commands must specify precise numerical parameters such as coordinates and feed rates. As a result, translating natural language instructions into executable CNC programs introduces additional challenges beyond those addressed in general natural language interface research. These challenges include the extraction of numerical parameters, the enforcement of safety constraints, and the generation of syntactically valid control instructions.

The next section therefore examines research on intent detection and semantic parsing methods, which form the core technologies used to transform natural language instructions into structured representations suitable for execution by technical systems.

3.4 Intent Detection and Semantic Parsing for Structured Commands

Interpreting natural language commands for technical systems typically requires the extraction of structured information from user instructions. In task-oriented language understanding systems, this process is often decomposed into two key subtasks: intent detection and slot filling. Intent detection identifies the overall goal of the user command, while slot filling extracts the relevant parameters required to execute the task.

This framework is widely used in conversational agents and task-oriented dialogue systems. However, it is equally relevant in technical command interpretation, where natural language instructions must be translated into structured machine actions. In industrial environments, structured command interpretation is particularly important because systems must extract precise parameters that can be used for machine control.

Several surveys have examined the evolution of intent detection and slot filling techniques in natural language understanding systems. Kamath et al. [KD18] provide a comprehensive survey of slot filling methods for natural language understanding. Their work reviews both traditional sequence labeling models and modern neural architectures used to identify semantic slots within user utterances. The survey highlights the growing importance of deep learning approaches for extracting structured information from natural language commands.

Early neural approaches to slot filling typically relied on recurrent neural networks and sequence labeling models. However, more recent work has focused on jointly modeling intent detection and slot filling in unified neural architectures. Goo et al. [GGH+18] propose a neural network architecture that performs joint intent detection and slot filling

using attention mechanisms. Their model improves performance by allowing the intent classification task to influence slot extraction and vice versa, enabling better contextual understanding of user instructions.

Building upon this idea, Qin et al. [QLC+21] introduce a co-interactive transformer architecture for joint intent detection and slot filling. Their model leverages transformer-based attention mechanisms to allow information sharing between the two tasks during the learning process. The proposed architecture demonstrates improved performance on several benchmark datasets by modeling the interaction between intent classification and slot extraction more effectively.

Another important direction in this research area focuses on improving the representation of semantic structures in language models. Wang et al. [WWR+21] explore syntax-aware transformer models that incorporate syntactic information into language understanding tasks. By encoding syntactic dependencies between words, their approach improves the extraction of structured semantic information from natural language.

In addition to general-purpose language understanding tasks, several studies have explored the application of semantic parsing techniques for translating natural language instructions into formal representations. Mishra et al. [MASA22] investigate methods for mapping natural language commands to structured task representations in automation systems. Their work highlights the importance of combining natural language understanding with structured output representations that can be executed by technical systems.

Table 3.4 summarizes representative studies on intent detection and semantic parsing for structured command interpretation.

Although these approaches have achieved strong performance in conversational systems and task-oriented dialogue, applying them directly to industrial command interpretation introduces additional challenges. Technical commands often contain numerical parameters, domain-specific terminology, and strict syntactic constraints that must be respected by the resulting structured representation.

In CNC machining environments, the extraction of parameters such as axis coordinates and feed rates is particularly important because these values directly determine machine behavior. Unlike conversational systems where minor interpretation errors may be tolerated, incorrect parameter extraction in CNC commands may lead to incorrect machine operations.

For this reason, many industrial systems combine machine learning based intent detection with deterministic validation mechanisms to ensure that extracted parameters satisfy operational constraints. The structured interpretation of natural language commands therefore requires both accurate semantic understanding and reliable parameter extraction mechanisms.

These challenges motivate the exploration of modern large language models for structured

Paper	Task	Approach	Main Contribution
Kamath et al. (2018) [KD18]	Slot filling	Survey of sequence labeling methods	Comprehensive overview of slot filling techniques in NLP systems
Goo et al. (2018) [GGH+18]	Intent + slot	Joint neural model with attention	Joint learning of intent detection and slot filling
Qin et al. (2021) [QLC+21]	Intent + slot	Co-interactive transformer	Modeling interactions between intent detection and slot extraction
Wang et al. (2021) [WWR+21]	Structured NLP	Syntax-aware transformer	Encoding syntactic structure for improved semantic extraction
Mishra et al. (2022) [MASA22]	Instruction parsing	Semantic parsing methods	Mapping natural language instructions to structured representations

Table 3.4: Representative studies on intent detection and semantic parsing for structured command interpretation.

command generation, which is discussed in the next section.

3.5 Large Language Models for Structured Code Generation

Large language models (LLMs) have recently become the dominant paradigm in natural language processing. These models are typically based on the transformer architecture and trained on massive text corpora using self-supervised learning objectives. By learning contextual representations of language at scale, LLMs are capable of performing a wide range of tasks including text generation, question answering, reasoning, and program synthesis.

One of the most significant capabilities of modern language models is their ability to generate structured outputs from natural language inputs. This ability has led to increasing interest in using LLMs for tasks such as code generation, database query generation, and structured information extraction. In these applications, the model must interpret natural language instructions and produce outputs that conform to strict syntactic and semantic constraints.

Recent surveys have highlighted the rapid growth of research on large language models and their applications. Wang et al. [WC23] provide a comprehensive review of large language model architectures and applications across various domains. Their study discusses the evolution of transformer-based models and emphasizes their potential for tasks requiring structured reasoning and generation.

One of the most prominent application areas for LLMs is code generation. Rajkumar et al. [RLB22] investigate the ability of large language models to generate executable code from natural language descriptions. Their evaluation demonstrates that while language models are capable of producing syntactically valid code, ensuring correctness and adherence to structural constraints remains challenging. This issue becomes particularly important in applications where generated code must be executed in real-world systems.

A key challenge in structured generation tasks is enforcing syntactic constraints during decoding. Scholak et al. [SSB21] introduce the PICARD framework, which constrains the decoding process of language models to ensure that generated outputs remain syntactically valid. The framework was originally developed for text-to-SQL generation tasks but illustrates a general strategy for improving the reliability of structured outputs generated by language models.

Another important research direction focuses on improving the extraction of structured information from natural language using large language models. Nan et al. [NZZ+23] investigate techniques for improving information extraction using transformer-based architectures. Their work demonstrates that incorporating structured output constraints can significantly improve the reliability of information extraction systems.

Similarly, Wang et al. [WSM+25] explore the use of large language models for slot filling and structured information extraction tasks. Their work shows that LLMs can achieve strong performance in extracting structured parameters from natural language inputs, particularly when trained or prompted with task-specific instructions.

Beyond information extraction, LLMs have also been applied to complex reasoning and task planning scenarios. Li et al. [LZW+24] examine how large language models can be used to perform structured reasoning across multi-step tasks. Their results suggest that LLMs can capture complex relationships between instructions and actions, enabling them to generate structured outputs for a variety of technical applications.

Table 3.5 summarizes representative studies on large language models applied to structured generation and information extraction tasks.

Despite their impressive capabilities, several challenges remain when applying large language models to structured generation tasks. Because LLMs generate outputs probabilistically, small deviations in numerical values or formatting may occur even when the semantic intent is correctly understood. In many applications such deviations may be acceptable, but in

Paper	Task	Approach	Main Contribution
Wang et al. (2023) [WC23]	LLM overview	Survey	Comprehensive review of large language model architectures and applications
Rajkumar et al. (2022) [RLB22]	Code generation	LLM evaluation	Evaluating LLM ability to generate executable code from natural language
Scholak et al. (2021) [SSB21]	Structured generation	Constrained decoding (PICARD)	Enforcing syntactic validity in structured outputs
Nan et al. (2023) [NZZ+23]	Information extraction	Transformer models	Improving structured information extraction and prompt design strategies with LLMs
Wang et al. (2025) [WSM+25]	Slot filling	LLM-based extraction	Using LLMs to structure and control model outputs for structured parameter extraction
Li et al. (2024) [LZW+24]	Task reasoning	Structured output frameworks	Benchmarking and evaluating structured output generation using large language models

Table 3.5: Representative studies on large language models for structured code generation and information extraction.

technical domains such as industrial automation they can lead to incorrect system behavior.

In industrial control systems, generated commands must satisfy strict structural constraints and numerical precision requirements. For example, CNC machine commands require precise coordinate values and feed-rate parameters that must remain within safe operational ranges. Ensuring that language models generate outputs that satisfy such constraints remains an open research problem.

These limitations have motivated recent research exploring hybrid approaches that combine language model based semantic interpretation with deterministic validation mechanisms. The next section examines recent studies that specifically investigate the use of large language models for CNC programming and G-code related tasks.

3.6 Large Language Models for CNC Programming and G-code Generation

Recent advances in large language models have also stimulated research on their application to manufacturing systems and CNC programming tasks. In particular, researchers have begun exploring whether natural language instructions can be automatically translated into executable G-code programs using machine learning and generative models. This research direction aims to simplify CNC programming by allowing operators to describe machining operations using natural language instead of manually writing low-level machine instructions.

Several studies have investigated the ability of large language models to interpret and manipulate CNC programs. Jignasu et al. [JMG+24] evaluate the capability of large language models to understand and reason about CNC programs. Their work examines whether language models can interpret G-code instructions and answer questions about machining operations. The results suggest that modern language models possess a basic understanding of CNC commands but may still struggle with complex reasoning involving numerical parameters.

Other research has explored the use of LLMs for directly generating G-code instructions from natural language descriptions. Choudki et al. [CKHT25] investigate prompt-based techniques for generating G-code programs from textual instructions. Their work demonstrates that prompt engineering can enable language models to produce machining commands, although the reliability of generated code depends heavily on the structure of the prompt and the constraints applied during generation.

Similarly, Kanimozhi et al. [KS24] conduct an exploratory study on the use of large language models for G-code generation tasks. Their experiments show that LLMs can successfully generate simple CNC commands but may produce inconsistent numerical parameters when instructions contain ambiguous language.

Another important research direction focuses on improving the reliability of generated CNC commands using specialized frameworks. Abdelaal et al. [ALE25] propose a framework called GLLM that integrates large language models with validation mechanisms for CNC code generation. The system attempts to ensure that generated commands satisfy structural constraints and machining safety requirements.

Few-shot learning approaches have also been explored for improving G-code generation performance. Pour et al. [PMN25] demonstrate that providing carefully selected example commands can significantly improve the quality of generated CNC programs. Their work highlights the importance of dataset design and prompt construction when applying language models to technical command generation.

Another promising research direction involves self-correcting language model systems. Shinn et al. [SCB+24] introduce the Reflexion framework, which enables language models to iteratively improve their outputs through self-reflection and error correction. Although the framework is not specifically designed for CNC programming, the concept of iterative correction is relevant for improving the reliability of generated machine instructions.

In addition to text-based reasoning, recent studies have explored multimodal approaches that combine visual information with language models. Yang et al. [YRST24] propose a framework that integrates visual-language models with reasoning capabilities to interpret structured tasks. Such approaches could potentially assist in validating generated CNC programs by analyzing machine states or visual feedback from machining processes.

Finally, Vsket et al. [ŠPB+25] examine the broader application of large language models in manufacturing systems and discuss their potential for supporting intelligent machining environments. Their work highlights how LLMs can assist in tasks such as program generation, documentation, and operator assistance.

Table 3.6 summarizes representative studies applying large language models to CNC programming and related manufacturing tasks.

Although these studies demonstrate promising progress in applying large language models to CNC programming tasks, several challenges remain. Many existing approaches rely heavily on prompt engineering or example-based learning, which may limit the reliability and reproducibility of generated commands. Additionally, the probabilistic nature of language model generation can lead to small numerical inconsistencies that may affect the correctness of CNC programs.

Another limitation is that most existing research evaluates language model performance primarily on code generation tasks rather than on strict industrial safety constraints. In CNC machining environments, generated commands must satisfy precise numerical limits and operational rules to ensure safe machine behavior.

These challenges highlight the importance of combining the semantic capabilities of large language models with deterministic validation mechanisms when designing natural language interfaces for CNC systems. The experiments presented in this thesis therefore compare structured intent–entity pipelines with large language model based approaches under strict numerical and safety constraints.

3.7 Research Gap

The literature reviewed in the previous sections demonstrates significant progress in several research areas related to this thesis. Studies on CNC systems and intelligent manufacturing

Paper	Application	Approach	Main Contribution
Jignasu et al. (2024) [JMG+24]	CNC program understanding	LLM reasoning	Evaluating LLM capability to interpret G-code programs
Choudki et al. (2025) [CKHT25]	G-code generation	Prompt engineering	Generating CNC commands from natural language prompts
Kanimozhi et al. (2024) [KS24]	G-code generation	Exploratory LLM study	Investigating LLM ability to produce CNC commands
Abdelaal et al. (2025) [ALE25]	CNC programming	GLLM framework	Integrating LLM generation with validation mechanisms
Pour et al. (2025) [PMN25]	G-code generation	Few-shot learning	Improving CNC code generation with example-based prompts
Shinn et al. (2024) [SCB+24]	Code generation reliability	Reflexion framework	Self-correcting LLM outputs through iterative feedback
Yang et al. (2024) [YRST24]	Multimodal reasoning	Vision-language models	Integrating multimodal reasoning for structured tasks
Vsket et al. (2025) [ŠPB+25]	Manufacturing systems	LLM applications	Overview of LLM usage in intelligent manufacturing environments

Table 3.6: Representative studies applying large language models to CNC programming and related manufacturing tasks.

have explored the integration of machine learning techniques for process monitoring and parameter optimization. Research on natural language interfaces has investigated methods for allowing users to interact with complex technical systems using everyday language. In parallel, advances in intent detection, semantic parsing, and large language models have enabled increasingly powerful techniques for translating natural language instructions into structured machine-readable representations.

Despite these advances, several important gaps remain when considering the problem of translating natural language operator instructions into executable CNC commands.

First, most research on intelligent manufacturing systems focuses on monitoring machining processes, optimizing machining parameters, or analyzing sensor data rather than on improving the interaction between human operators and CNC controllers. While these approaches contribute to the development of intelligent manufacturing environments, they do not directly address the problem of allowing operators to express machining instructions using natural language.

Second, existing work on natural language interfaces and task-oriented dialogue systems

typically targets conversational assistants, robotic systems, or general task planning applications. These systems often operate with symbolic task representations or logical action descriptions. In contrast, CNC programming requires the generation of precise numerical parameters such as axis coordinates and feed rates, which directly control machine movements. As a result, methods developed for conversational systems cannot be directly applied to CNC programming without additional mechanisms for parameter extraction and validation.

Third, research on intent detection and semantic parsing has primarily focused on improving the accuracy of language understanding in conversational systems. Although these techniques are highly relevant for interpreting user commands, they are rarely evaluated in industrial automation contexts where extracted parameters must satisfy strict operational constraints.

Recent studies exploring the application of large language models to CNC programming have demonstrated promising capabilities in generating G-code commands from natural language instructions. However, many of these approaches rely heavily on prompt engineering or example-based generation strategies. Such methods often lack deterministic guarantees and may produce small numerical inconsistencies in generated commands. In industrial environments where machine instructions directly control physical equipment, even minor deviations in numerical parameters may lead to incorrect machining operations.

Furthermore, most existing research evaluates language model performance primarily using general code generation metrics rather than safety-oriented evaluation criteria that are necessary for industrial deployment. Ensuring that generated commands remain within safe operational limits and satisfy structural constraints remains an open challenge.

The research presented in this thesis aims to address these gaps by investigating the translation of natural language operator instructions into executable CNC G-code commands under strict numerical and safety constraints. In particular, this work compares two different paradigms for structured command interpretation: a deterministic intent–entity pipeline based on a natural language understanding framework, and large language model based approaches for structured command generation. By evaluating these approaches under consistent safety and validation criteria, the thesis contributes to a better understanding of the strengths and limitations of modern language models in industrial control applications.

Chapter 4

Analysis and Design

4.1 Dataset

At the beginning of the project, Grinding Technology S.r.l. provided a reference document containing basic G-code rules and example mappings between natural language commands and CNC instructions. This document describes the structure of ISO-standard G-code commands used in grinding operations, with particular attention to positioning movements, cutting operations, and homing procedures.

The reference material was used as a specification to define system constraints, including supported G-code commands, coordinate conventions, and safety requirements. It was not used directly as a dataset, but rather as a guideline to ensure that all generated data and system outputs reflected realistic CNC machine behavior.

Based on the technical specifications provided by the company, an additional 150 sentence–command pairs were manually constructed by the author in both English and Italian. Each pair was subsequently reviewed and validated for syntactic correctness and operational consistency by Dr. Spingardi to ensure compliance with the defined G-code constraints and machining semantics. The inclusion of bilingual examples increased lexical and syntactic variability while preserving semantic equivalence with realistic operator instructions.

Initial experiments using a subset of approximately 150 manually created sentences revealed that the dataset was not sufficiently large or diverse to capture the full variability of natural language operator commands. To address this limitation, feedback was requested from domain experts at Grinding Technology S.r.l.

The expert review highlighted issues introduced during data augmentation, such as inappropriate synonym substitutions that altered the original meaning of certain commands. These issues were corrected, and explicit operational constraints were defined for the main

machine parameters, including longitudinal table movement (X), wheel plunge depth (Z), and feed rate (F).

Each sentence in the dataset is associated with an intent, a set of entities, and a corresponding G-code command. The entities include spatial coordinates (X and Z) and feed-rate information (F), which directly influence the generated G-code. The machining operation type and cutting speed may be expressed either explicitly through numeric values or implicitly through linguistic descriptors.

Data augmentation techniques, such as rephrasing commands and varying numerical values while preserving semantic meaning, were applied to expand the dataset. After refinement and validation, the final dataset consisted of approximately 400 sentence–G-code pairs used for training and evaluation.

Three intent types are defined in the dataset: *grind_linear*, *pure_positioning*, and *homing*, corresponding respectively to the G-code commands G01, G00, and G28.

Based on the problem analysis and dataset characteristics, the system is required to accept a natural language command as input and generate a valid G-code instruction as output. The system must correctly identify the intent of the command and extract relevant entities such as coordinates and feed-rate information.

The generated G-code must respect CNC operational constraints and follow the structure defined in the initial specifications. Only a limited set of commands is supported, namely rapid positioning, linear grinding operations, and homing procedures. The system must distinguish between explicit numeric feed-rate values and descriptive feed-rate expressions and reflect them correctly in the generated G-code.

Qualitative cutting descriptors (e.g., rough cut, light cut, fine cut, and spark-out) were associated with specific feed-rate ranges to ensure a consistent mapping between natural language expressions and machine parameters. Intent definitions were also refined to clearly separate homing and safety movements from pure positioning operations, while reserving cutting-related actions for linear grinding commands.

Beyond numerical constraints, validation rules were applied to ensure internal consistency between natural language commands, annotations, and generated G-code. Linear grinding operations were required to include a valid feed rate and a negative Z value corresponding to material removal, while homing commands were restricted to reference movements. Pure positioning operations were constrained to rapid movements without feed-rate specification.

The operational constraints and intent definitions used for dataset construction are summarized in Tables 4.1, 4.2, and 4.3. Table 4.1 defines the numerical limits for machine parameters, while Table 4.2 presents the mapping between qualitative linguistic descriptors and feed-rate values. Finally, Table 4.3 summarizes the intent categories and their associated G-code commands.

Parameter	Description	Allowed Range
X	Longitudinal table movement	-40 to $+50$ mm
Z	Wheel plunge depth	$+10$ to -20 mm
F	Feed rate	5 to 30 mm/min

Table 4.1: Operational constraints used for dataset construction

Linguistic descriptor	Cut type	Feed rate (F)	Notes
spark-out, finishing, with precision, final pass	Spark-out / finishing	5–6	Final polishing pass
fine, slowly, carefully	Fine cut	8–12	Low feed, reduced material removal
light, superficial	Light cut	15–20	Medium feed, shallow depth
rough, deep, heavy	Rough cut	25–30	High feed, aggressive removal
(no descriptor)	Standard	15	Default neutral case

Table 4.2: Mapping between linguistic descriptors, cut types, and feed-rate values

Intent Category	Description	Associated G-code
Homing / Safety moves	Return to reference or safe position	G28
Pure positioning	Fast positioning without cutting	G00
Linear grinding	Material removal operations	G01
Auxiliary actions (future)	Non-linear or auxiliary commands	G02, G03, G04, etc.

Table 4.3: Definition and scope of intent categories

The dataset construction process therefore required iterative validation cycles. After each augmentation phase, the generated sentence–G-code pairs were manually reviewed to ensure consistency between linguistic expressions and numeric parameters. Particular attention was paid to cases where subtle wording changes could alter the machining semantics, such as differences between positioning and cutting operations.

The dataset contains both English and Italian sentences to reflect realistic operator usage conditions. The structured Rasa-based baseline was evaluated on both language subsets. However, large language model experiments were conducted only on the English subset to ensure controlled comparison under consistent training and decoding conditions.

The Rasa-based structured intententity model was trained and evaluated on both the English and Italian subsets of the dataset. This allowed assessment of cross-lingual robustness within a schema-based framework.

In contrast, prompt-based and fine-tuned LLM experiments were conducted exclusively on the English subset. This choice was motivated by the fact that most instruction-tuned large language models are primarily optimized and benchmarked in English, and controlled evaluation required consistent linguistic conditions across architectures.

Operators in industrial environments may switch between languages or use hybrid terminology. By incorporating multilingual variability, the dataset better reflects realistic usage conditions and increases the robustness of both structured and generative models evaluated later in this thesis.

Furthermore, the distribution of intent classes was balanced to avoid bias toward a single machining operation. Grinding commands (G01), positioning commands (G00), and homing commands (G28) are all sufficiently represented, ensuring that model performance cannot be artificially inflated by class imbalance.

4.1.1 Dataset Expansion with Explicit Feed-Rate Values

In addition to qualitative cutting descriptions, an extended set of sentences was introduced to include explicit numerical feed-rate values. Approximately 100 additional sentence–G-code pairs were created in which the feed rate was directly specified in the natural language command (e.g., *"Move the wheel to X4 Z-3 at F8"*).

This extension reflects realistic operator behavior, where feed-rate values are often explicitly provided rather than inferred. Including explicit feed-rate examples reduced ambiguity during training and enabled a clearer separation between feed-rate extraction and feed-rate inference mechanisms.

The introduction of explicit feed-rate values also allowed a more precise evaluation of

numeric extraction performance. When feed rates are explicitly stated in the input, the correct output requires exact reproduction of the numeric value rather than inference from qualitative descriptors. This creates a clearer benchmark for assessing the numerical stability of both the Rasa-based and LLM-based approaches.

Moreover, the presence of explicit feed-rate examples reduces reliance on handcrafted descriptor mappings and enables a more realistic simulation of industrial operator input patterns. In many practical scenarios, experienced operators directly specify feed-rate values based on production requirements or material characteristics. Including such examples therefore enhances the ecological validity of the dataset.

With the final dataset consisting of 500 sentence–G-code pairs, the training and evaluation process benefits from increased linguistic diversity and improved numeric coverage across the allowed parameter ranges.

4.2 Design

4.2.1 Structured Intermediate Representation

A central design choice in this thesis was the use of a structured intermediate representation between natural language input and final G-code generation. Rather than treating command synthesis as a purely direct mapping from text to executable code, the task was conceptually decomposed into two stages: semantic interpretation and deterministic synthesis.

The semantic interpretation stage identifies the intended operation and the parameters required to execute it. In the present problem, this representation is expressed through the combination of an intent label and a set of entities. The intent captures the general command class, such as grinding, positioning, or homing, while the entities encode the relevant machine parameters, including axis coordinates and feed-rate information.

This intermediate representation provides several advantages. First, it separates language understanding from CNC command formatting, making the overall architecture easier to interpret and debug. Second, it enables validation rules to be applied before final code generation, reducing the risk of unsafe or incomplete outputs. Third, it creates a common conceptual interface across different architectures, including rule-based pipelines and large language model based systems.

In the Rasa-based approach, this intermediate representation is explicitly modeled through intent classification and entity extraction. In the LLM-based approaches, the same structure is either implicitly inferred by the model or explicitly reconstructed through prompt design or staged orchestration. In both cases, the presence of a structured representation helps ensure

that natural language interpretation remains aligned with the deterministic requirements of CNC command generation.

The adoption of this intermediate layer also reflects the practical nature of industrial control tasks. Operators typically communicate intentions and parameters at a conceptual level, whereas CNC controllers require rigid syntactic commands. The intermediate representation therefore serves as a bridge between human-level instruction and machine-level execution.

4.2.2 Rasa-Based Approach

After creating the dataset, the objective was to compare a traditional NLU-based approach with large language models in order to evaluate their performance. The experimentation therefore started with a Rasa-based system, which served as an initial structured baseline for natural language to G-code translation.

At the early stage of the project, the goal was to translate natural language operator commands into valid CNC G-code instructions. The Rasa framework was selected due to its support for intent classification, entity extraction, and rule-based dialogue management. These characteristics make Rasa particularly suitable for structured and safety-critical domains such as CNC machining, where predictable behavior and explicit control are required.

The first version of the Rasa-based intententity model was trained using an initial subset of the dataset described in Chapter 4.1, containing approximately 150 annotated sentencecommand pairs. Preliminary experiments showed acceptable performance on simple and frequently occurring command patterns. However, performance degraded when confronted with variations in phrasing, alternative word orders, or previously unseen numeric combinations. These observations indicated limited generalization capacity under the initial data distribution and motivated further dataset expansion and refinement.

Rasa-Based Interactive Interface

The Rasa-based system was implemented using a structured intent–entity architecture. Intent classification determines the type of CNC operation, while entity extraction identifies numerical parameters and semantic descriptors.

A custom action module was implemented to handle CNC-specific logic. This module extracts X, Z, and F values from the user input, infers feed-rate categories from linguistic descriptors when explicit values are missing, validates numerical parameters against predefined safe ranges, and generates the final G-code instruction.

Dialogue management was implemented using rule-based policies to preserve deterministic

behavior appropriate for a safety-critical application.

When mandatory parameters are missing or under-specified, the system supports multi-turn clarification. In particular, if a grinding command lacks an explicit feed-rate value and the qualitative descriptor does not uniquely determine a numeric value, the chatbot requests additional input from the user before generating the final G-code instruction.

For example, when a qualitative descriptor such as “deep cut” is detected, the system maps it to a feed-rate category (e.g., F_fast) and requests explicit confirmation of a valid numeric value within the allowed range before synthesis. Only after receiving compliant input does the system produce the final G-code command.

Figure 4.1 illustrates an example interaction from the implemented system. The model detects the intent and partial parameters, identifies the feed-rate category, prompts the user for clarification, and synthesizes the validated G-code instruction once sufficient information is available.

This clarification mechanism reduces the risk of parameter hallucination and prevents the generation of incomplete or unsafe commands, which is particularly important in safety-critical CNC environments.

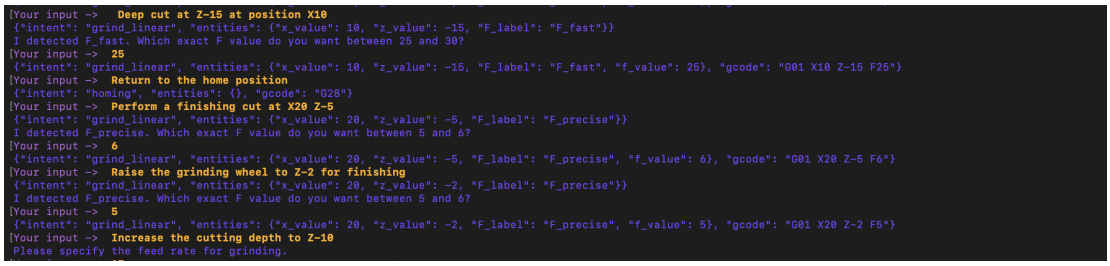

Figure 4.1: Example of multi-turn clarification dialogue in the Rasa-based system. The system detects feed-rate categories and requests explicit numeric confirmation before generating G-code.

Initially, Rasa was employed primarily as an NLU engine implementing the intent–entity pipeline and deterministic G-code synthesis. After validating the core architecture, the system was exposed through an interactive command-line interface, allowing users to issue natural language commands and receive generated G-code in real time. This deployment enabled practical testing of clarification behavior and parameter validation under realistic usage conditions.

Handling Explicit and Implicit Feed-Rate Specifications

The Rasa-based system handles both explicit and implicit feed-rate specifications. When the operator provides a numerical feed-rate value (e.g., *“Move the wheel to X4 Z-3 at F8”*),

the value is directly extracted as an entity and validated before inclusion in the generated G-code instruction.

In contrast, when no explicit feed-rate value is provided, the system infers the feed rate from qualitative cutting descriptors using predefined feed-rate categories and numeric ranges. This distinction supports realistic operator input while maintaining a clear separation between feed-rate extraction and feed-rate inference.

To improve numerical consistency, a rule-based feed-rate normalization strategy was introduced. Linguistic cutting descriptors are mapped to predefined feed-rate categories associated with fixed numeric ranges reflecting safe industrial practice. Rather than assigning arbitrary values within a range, the system enforces consistent mapping rules to ensure reproducible command generation.

This refinement strengthens the deterministic constraint layer of the architecture by reducing ambiguity in parameter assignment while preserving safety compliance.

Overall, the Rasa-based system provides a structured and deterministic baseline for translating natural language operator commands into CNC G-code instructions. Its layered designcombining intent classification, entity extraction, deterministic validation, and controlled multi-turn clarificationmakes it well suited to safety-critical industrial applications. However, the reliance on handcrafted normalization and rule-based parameter handling limits scalability as linguistic variability increases, motivating the exploration of large language model-based approaches.

4.2.3 LLM-Based Approach

This section describes the conceptual design of the LLM-based approach for translating natural language CNC commands into executable G-code.

The primary design objective is to generate a single, safe, and syntactically valid G-code command that accurately reflects the users intent while respecting all operational constraints of the CNC machine.

Given a natural language command expressed by a machine operator, the system must generate exactly one valid G-code instruction corresponding to a supported CNC operation. The output command must belong to one of the following motion types: `G00` (rapid positioning), `G01` (linear grinding with feed-rate), or `G28` (homing).

Formally, the task is modeled as a conditional generation problem:

$$\text{G-code} = f_{\text{LLM}}(\text{Natural Language Command})$$

where the generated output must satisfy syntactic correctness, numerical validity, and

operational safety constraints.

Because the generated G-code is intended to control a real industrial machine, safety constraints are treated as hard requirements. Each output must satisfy the following conditions:

- The output must consist of exactly one line of G-code.
- The command must start with one of the allowed instructions: `G00`, `G01`, or `G28`.
- All numerical parameters must lie within predefined safe ranges:

$$X \in [-40, 50], \quad Z \in [-20, 10], \quad F \in [5, 30].$$

These constraints are explicitly encoded in the prompting strategy and verified during evaluation to ensure that no unsafe or invalid commands are produced.

Two complementary paradigms for using LLMs were investigated in this work:

- **Prompt-based inference**, where a pre-trained LLM is instructed through carefully designed prompts without any task-specific training.
- **Supervised fine-tuning**, where a pre-trained sequence-to-sequence model is adapted to the CNC domain using paired natural language and G-code examples.

Prompt-Based Design

In the prompt-based setting, the LLM is instructed to directly infer intent and numerical parameters from the input sentence and to generate a valid G-code command. The prompt design emphasizes:

- explicit specification of allowed G-code commands;
- clear instruction to output only a single G-code line;
- guidance for handling feed-rate inference when qualitative descriptors (e.g., “slowly”, “rough”) are used.

This design treats the LLM as an implicit joint model for intent recognition, parameter extraction, and command synthesis.

Fine-Tuning Design

For supervised learning, the task is formulated as a sequence-to-sequence mapping between natural language commands and their corresponding G-code instructions. Each training example consists of a sentence–command pair, allowing the model to learn:

- consistent G-code formatting;
- implicit intent classification through command prefix prediction;
- extraction and placement of numerical parameters.

Fine-tuning shifts part of the constraint handling from the prompt into the model parameters, reducing sensitivity to prompt wording and improving output stability. This approach is particularly effective when combined with a refined dataset that enforces consistent annotation rules and explicit feed-rate normalization.

Evaluation-Oriented Design Considerations

The LLM-based design explicitly distinguishes between strict correctness and functional correctness. While exact string matching is used as a primary metric, it penalizes minor numerical differences that may still be operationally valid.

For this reason, intent accuracy and safety validity are considered complementary evaluation dimensions, enabling a more nuanced assessment of model behavior in an industrial context.

Chapter 5

Implementation

The implementation phase reflects the refined dataset described in Section 4.1, including the introduction of explicit numeric feed-rate examples. This refinement required adjustments to entity extraction logic and numeric validation strategies in both the structured and generative pipelines.

The implementation goal was to construct two complete systems under identical operational constraints:

- A structured NLU pipeline using Rasa;
- A generative pipeline using large language models (LLMs).

To ensure fairness and safety, both systems share a unified post-processing and validation module that programmatically enforces the operational constraints defined in Section 4.1.

The validation layer verifies numeric ranges, intent-dependent parameter requirements, and output formatting before a command is accepted as valid.

and requiring that every output:

- contains exactly one line,
- begins with G00, G01, or G28,
- satisfies intent-dependent parameter requirements.

This separation between interpretation and validation ensures that no unsafe command can bypass constraint checking.

5.1 Processing Workflow

Although the structured and generative pipelines rely on different language interpretation mechanisms, both follow a common processing workflow. The workflow defines how a natural language instruction is transformed into a validated CNC command.

The processing sequence consists of four main stages:

1. Natural language interpretation
2. Structured parameter extraction
3. G-code synthesis
4. Deterministic validation

In the first stage, the operator instruction expressed in natural language is interpreted by the language understanding module. In the Rasa-based system, this stage consists of intent classification and entity extraction. In the LLM-based systems, interpretation is performed through prompt-driven sequence generation or structured extraction chains.

The second stage converts the interpreted information into a structured parameter representation. This representation contains the predicted intent and the extracted parameters required to execute the command, such as axis coordinates and feed-rate values. The structured representation acts as an intermediate interface between language interpretation and CNC command synthesis.

In the third stage, the structured parameters are converted into an executable G-code instruction. This synthesis step enforces the correct ordering of command tokens and applies intent-dependent formatting rules. For example, linear grinding commands must include axis coordinates and feed-rate parameters, while homing commands contain no positional parameters.

The final stage performs deterministic validation of the generated command. Numeric values are checked against predefined safety ranges, command structure is verified, and parameter consistency is enforced. Commands failing any validation rule are rejected or trigger a clarification request before execution.

By maintaining a consistent workflow across all pipelines, the system ensures that the experimental comparison between Rasa-based and LLM-based approaches is conducted under identical operational conditions.

5.2 Rasa-Based Pipeline Implementation

The Rasa-based system was implemented using Rasa Open Source with a deterministic intententity architecture.

NLU Configuration

The NLU pipeline performs:

- Intent classification among *grind_linear*, *pure_positioning*, and *homing*;
- Entity extraction for X, Z, F, and qualitative feed-rate descriptors.

Training data consist of 500 annotated sentences. The inclusion of 100 explicit F-number examples increased numeric entity distribution and reduced reliance on descriptor inference.

Regular expressions were integrated into the pipeline to ensure robust extraction of:

- Signed numeric coordinates (e.g., Z-3);
- Explicit feed-rate values (e.g., F12);
- Numeric tokens expressed with optional spaces or punctuation.

The Rasa-based pipeline was also configured to support bilingual input. Training data included both English and Italian operator commands, allowing the system to learn intent patterns and parameter expressions across the two languages. Preliminary functional tests on selected Italian examples showed that the chatbot could correctly identify intents and extract relevant parameters from Italian inputs. In particular, the system correctly handled Italian homing and grinding commands and preserved the same clarification behavior used in English when an exact feed-rate value was required. An example of this behavior is reported in Appendix A.5. However, no systematic quantitative evaluation was conducted on the Italian subset, and bilingual performance was therefore assessed only qualitatively.

Custom CNC Action and Deterministic Synthesis

A custom Python action encapsulates CNC-specific logic. After receiving predicted intent and extracted entities, the action executes the following sequence:

1. Verify mandatory parameters based on intent.
2. Apply feed-rate resolution logic.
3. Enforce numeric range validation.
4. Construct a formatted G-code string.
5. Perform final validation before returning output.

Intent-dependent constraints are enforced:

- G01 requires X, Z, and F;
- G00 requires X and Z but no F;
- G28 requires no axis parameters.

5.3 Feed-Rate Resolution Mechanism

Feed-rate handling is implemented as a prioritized decision hierarchy.

Step 1: Explicit numeric extraction

If an F-value is explicitly present in the input (introduced in the additional 100 dataset examples), it is directly extracted and validated within the allowed range.

Step 2: Descriptor-based inference

If no explicit F is detected but a qualitative descriptor is present, a deterministic mapping table assigns a canonical feed-rate value corresponding to predefined industrial categories.

Step 3: Clarification fallback

If neither explicit nor inferable information is available for a G01 command, the system triggers a multi-turn dialogue requesting clarification.

This hierarchy ensures safety and reflects the dataset construction logic described in Chapter 4.1.

5.3.1 Quantitative features of the developed Rasa code

The Rasa-based implementation combines configuration files, training data, and a custom action module to realize deterministic CNC command generation.

The system includes the following components:

- **NLU training data:** 111 lines in `nlu.yml`, containing annotated examples for three intent classes (*grind_linear*, *pure_positioning*, *homing*) and associated entities (X, Z, F, and qualitative descriptors).
- **Dialogue rules:** 24 lines in `rules.yml`, defining deterministic clarification and response behavior.
- **Domain definition:** 47 lines in `domain.yml`, specifying intents, entities, slots, and response templates.
- **Pipeline configuration:** 29 lines in `config.yml`, defining the NLU architecture and entity extraction components.
- **Custom CNC action module:** 183 lines of Python code in `actions.py`, implementing feed-rate resolution, parameter validation, intent-dependent constraints, and G-code synthesis.

Overall, the structured Rasa implementation comprises 394 lines of configuration and logic code, excluding auxiliary evaluation scripts and model artifacts.

The custom action module represents the core deterministic layer of the system. It enforces safety constraints, numeric range validation, hierarchical feed-rate handling, and final output verification before returning the generated G-code instruction.

This quantitative breakdown highlights the engineering effort required to implement rule-based constraint enforcement and safe industrial command generation within a structured NLU framework.

5.4 LLM-Based Pipeline Implementation

The LLM-based implementation models the task as conditional sequence generation:

$$G = f_\theta(NL)$$

where the model must produce a syntactically valid and numerically constrained G-code command.

Two paradigms were implemented.

Prompt-Based Inference

Prompt-based inference was implemented using LLaMA-3.1 70B via the Groq API. The model was instructed through a structured prompt designed to enforce deterministic output behavior, schema consistency, and CNC safety constraints.

The prompt explicitly encodes: - a fixed JSON output schema, - a priority-ordered intent decision hierarchy, - descriptor normalization rules, - feed-rate resolution logic, - strict G-code formatting constraints.

The complete prompt template used during evaluation is reported in Appendix A.4 to ensure full reproducibility.

Supervised Fine-Tuning (FLAN-T5)

Fine-tuning was performed using FLAN-T5 Base within the Hugging Face Transformers framework.

Each training sample consists of:

- Input: Natural language sentence;
- Target: Corresponding G-code command.

Preprocessing steps include:

- Lowercasing and normalization;
- Tokenization;
- Padding and truncation;
- Alignment of decoder targets.

The addition of explicit F-number examples improved numeric reproduction accuracy by providing direct supervision for feed-rate copying rather than semantic inference alone.

5.4.1 Quantitative Features of the Developed FLAN-T5 Code

The supervised fine-tuning pipeline was implemented in a Google Colab environment using the Hugging Face Transformers framework.

The implementation consists of 16 code cells comprising approximately 202 lines of Python code. These lines include data preprocessing, tokenization, training configuration, evaluation procedures, and post-generation validation logic.

The notebook implements:

- Dataset loading and formatting into sequence-to-sequence inputoutput pairs;
- Tokenization and alignment of encoder and decoder targets;
- Configuration of training arguments (optimizer, learning rate, batch size, epochs);
- Fine-tuning of the FLAN-T5 Base model;
- Controlled decoding and output normalization;
- Post-generation validation enforcing operational CNC constraints.

Although model fine-tuning is supported by the Transformers library, a substantial portion of the implementation is dedicated to data preprocessing, evaluation, and safety validation. This reflects the engineering effort required to adapt a general-purpose instruction-tuned model to a safety-critical industrial code generation task.

5.5 Structured LLM Pipeline (LangChain)

To reduce numeric instability, a modular LLM pipeline was implemented using LangChain.

This architecture separates:

1. Intent prediction;
2. Entity extraction;
3. Deterministic synthesis;
4. Final validation.

This reduces error propagation compared to single-step generative prompting and improves entity-level consistency.

5.5.1 Quantitative features of the developed Langchain code

The structured LLM pipeline implemented using LangChain was developed as a modular orchestration framework integrating multiple constrained LLM interactions and deterministic validation layers.

The implementation was developed in a dedicated Google Colab notebook consisting of 18 executable code cells and approximately 330 lines of Python code, excluding auxiliary visualization and evaluation scripts.

The architecture is composed of the following main components:

- A dedicated intent classification chain;
- A structured entity extraction chain returning JSON-formatted outputs;
- A deterministic G-code synthesis module;
- A numeric validation and constraint enforcement layer;
- A structured parsing and error-handling mechanism to detect malformed LLM outputs.

Unlike single-step prompt-based inference, the LangChain pipeline explicitly decomposes the generation process into sequential stages. Intermediate outputs are parsed, validated, and reformatted before being passed to subsequent modules. This required implementation of:

- Custom prompt templates for each stage;
- JSON schema validation to ensure numeric consistency;
- Consistency checks between extracted parameters and final G-code;
- Intent-dependent parameter verification;
- Controlled fallback logic in case of malformed or incomplete LLM outputs.

A significant portion of the implementation effort was dedicated to mitigating typical large language model instabilities, including numeric drift, inconsistent formatting, and partial JSON generation. Structured parsing and validation were necessary to ensure that the probabilistic generation process produced industrially safe commands.

Overall, the LangChain implementation reflects the engineering complexity required to stabilize generative models for safety-critical CNC control tasks. While less rule-heavy than the Rasa-based deterministic system, the structured LLM pipeline required explicit orchestration logic and multi-stage constraint enforcement to achieve reliable behavior.

Unified Validation and Safety Layer

A shared validation module ensures that all outputswhether from Rasa or LLMare evaluated under identical criteria.

For the structured Rasa system, validation logic is integrated within the custom action module (actions.py), which consists of 183 lines of Python code. A substantial portion of this module is dedicated to deterministic constraint enforcement and numeric validation.

The validation layer performs:

- Regular-expression-based extraction of numeric parameters (X, Z, F);
- Enforcement of allowed G-code prefixes (G00, G01, G28);
- Intent-dependent parameter constraints (e.g., G01 requires X, Z, and F);
- Feed-rate range validation using predefined industrial intervals;
- Consistency verification between extracted numeric values and generated G-code tokens;
- Hierarchical feed-rate resolution logic with descriptor-based mapping;
- Multi-turn clarification when feed-rate information is incomplete.

Feed-rate validation is implemented through explicit range tables (FEED_RATE_RANGES), and numeric values are programmatically checked before G-code synthesis. Commands violating intent-dependent or numeric constraints are not executed and instead trigger clarification or rejection.

For LLM-based approaches, generated outputs are passed through the same Python-based validation functions after parsing. Additional safeguards were implemented to detect malformed JSON, multi-line outputs, invalid command prefixes, and numeric inconsistencies introduced by probabilistic generation.

By centralizing constraint enforcement into a shared deterministic layer, architectural differences between structured NLU and generative LLM approaches do not affect safety guarantees. This design establishes a clear separation between probabilistic language interpretation and industrial-grade CNC command validation.

Implementation Environment

Rasa experiments were executed locally using Python and the Rasa Open Source framework.

Fine-tuning experiments were conducted using Hugging Face Transformers.

Prompt-based inference was executed via API access to LLaMA-3.1 70B.

LangChain-based orchestration was implemented in Python.

All experiments used fixed dataset splits and consistent evaluation scripts to ensure reproducibility and comparability across approaches.

The implementation integrates structured NLP techniques and generative AI models within a unified safety-controlled framework. This enables a controlled experimental comparison while ensuring that no generated CNC instruction violates predefined operational constraints.

Chapter 6

Experiments

6.1 Rasa-Based Approach

Experimental Setup and Data Split

All the experiments in Rasa were done on the same split of the dataset to compare different system settings. The data was split into training and test sets, with the test set kept constant for all experiments. Both the initial evaluation and the evaluation done after normalization of the feed rate were done on the same test data. This ensures that any differences in performance observed are due to the differences in the handling of the feed rate logic and not due to differences in the evaluation data.

Initial Rasa Evaluation Results

The first assessment of the Rasa system was done using the first version of the dataset and the logic for inferring the feed rate. The evaluation criteria included the accuracy of intent classification, performance of entity extraction, and correctness of the produced G-code. The component-wise and joint accuracies achieved in this initial setting are reported in Figure 6.1, and the numerical results are summarized in Table 6.1. The results indicate that the accuracy of intent classification was high, which means that the system was able to distinguish between the different types of CNC operations. The performance of entity extraction was also good for the X and Z coordinates. However, the accuracy of G-code and joint accuracy were not as good, mainly because of the irregularity in the assignment of the feed rate.

The use of qualitative accuracy labels reflects the fact that performance values varied

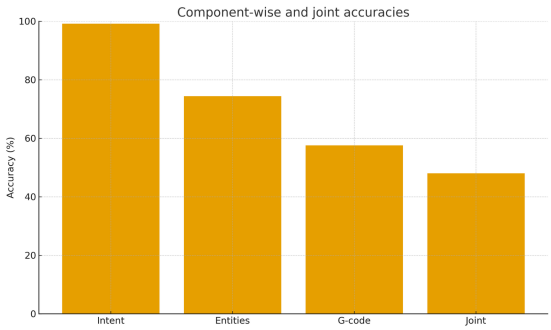

Figure 6.1: Initial component-wise and joint accuracies of the Rasa-based system before feed-rate normalization

Component	Accuracy (%)
Intent	High
Entities	Moderate–High
G-code (Exact Match)	Moderate
Joint Accuracy	Lower

Table 6.1: Summary of initial evaluation results for the Rasa-based system.

across different evaluation runs while preserving the relative comparison between system components.

Feed-Rate Normalization Rules

Analysis of the initial results showed that the majority of the G-code mismatches were due to feed rate differences, and not due to incorrect classification of the intent or the extraction of the axis values. In particular, qualitative cutting descriptors like light, fine, or rough were assigned to different values of the numerical feed rate, even though more than one value was operationally valid. To tackle this problem, a rule-based feed-rate normalization approach was proposed, which linked linguistic cutting descriptors with pre-defined feed-rate values and numerical ranges, as depicted in Table 6.2.

Cut Type Descriptor	Feed-Rate Category	Feed-Rate Range
Spark-out, finishing, final pass	F_{precise}	5–6
Fine, slowly, carefully	F_{slow}	8–12
Light, superficial	F_{medium}	15–20
Rough, deep, heavy	F_{fast}	25–30

Table 6.2: Rule-based mapping between cutting descriptors and feed-rate categories.

Rasa Evaluation Results After Feed-Rate Normalization

After the introduction of the feed rate normalization rules and the improvement of the dataset to ensure proper alignment between the linguistic descriptions and the parameters, the system was re-evaluated. The refined component-wise and joint accuracies are depicted in Figure 6.2 after applying the proposed rules. In comparison to the first evaluation, there is a noticeable improvement in the accuracy of the G-code and the joint accuracy, while the performance of the intent classification task remained stable.

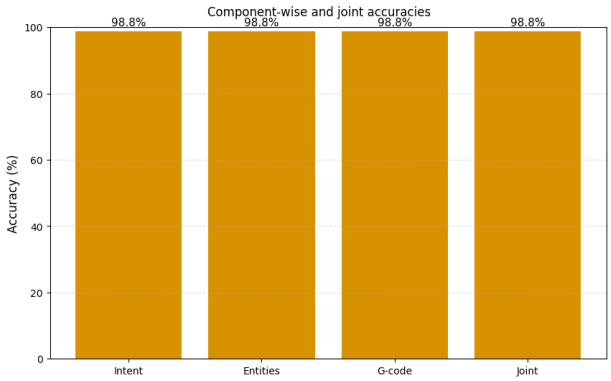

Figure 6.2: Component-wise and joint accuracies of the Rasa-based system after feed-rate normalization

Despite the improvement in exact-match accuracy, some discrepancies remained due to small numerical variations within acceptable feed-rate ranges, motivating the use of functional correctness as a complementary evaluation criterion.

Intent Classification Performance

Intent classification performance was evaluated using standard precision, recall, and F1-score metrics. Table 6.3 reports the classification results for the three supported intent classes.

The results show consistently high performance across all intents, with F1-scores close to 0.99 for linear grinding commands and above 0.98 for both homing and pure positioning commands. This confirms that the Rasa-based system is able to reliably distinguish between different types of CNC operation.

Figure 6.3 shows the distribution of exact G-code matches and mismatches produced by the Rasa-based system. The distribution further highlights that most incorrect predictions are not due to intent misclassification, but rather to parameter-level discrepancies, particularly in feed-rate assignment.

Qualitative examples of system outputs, including multi-turn interactions and G-code generation, are reported in the Appendix A.3.

Intent	Precision	Recall	F1-score	Support
grind_linear	0.99	0.99	0.99	305
homing	0.98	0.98	0.98	43
pure_positioning	0.98	0.98	0.98	152
Overall			0.99	500

Table 6.3: Intent classification performance of the Rasa-based system.

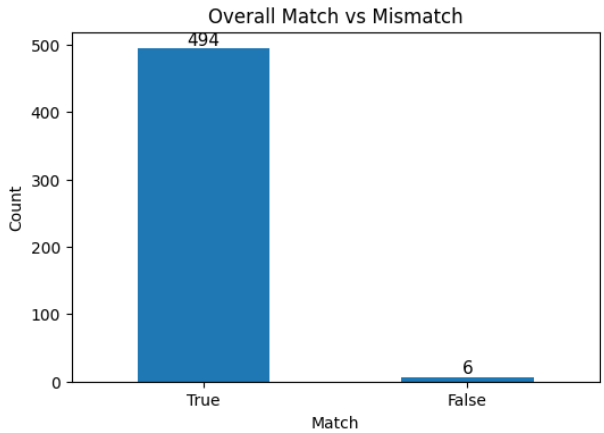

Figure 6.3: Exact G-code match versus mismatch counts for the Rasa-based system

Error Analysis of the Rasa-Based System

An analysis of the remaining errors revealed that the majority of of incorrect predictions were not caused by intentional misclassification but by parameter-level inconsistencies. In particular, feed-rate assignment accounted for most mismatches in the exact G-code comparison, even when the generated command was operationally valid.

Errors related to missing or incorrect X and Z coordinates were rare and intent misclassification occurred infrequently. Early experiments conducted on the initial dataset showed a higher incidence of such errors, including misclassification between grinding and positioning commands. These issues were substantially reduced after the refinement of the data set and the introduction of feed-rate normalization rules.

Overall, the error analysis confirms that the limitations of the Rasa-based system are primarily related to the rigidity of rule-based parameter handling rather than to deficiencies in intent recognition.

6.2 LLM-Based Approach

This section presents the experimental results obtained using large language model (LLM)-based approaches for translating natural language commands into CNC G-code instructions. All experiments were conducted under the same safety constraints as for the Rasa-based system. Each generated output was required to consist of a single G-code line, begin with a supported command (G00, G01, or G28) and respect predefined numerical ranges for X, Z and feed-rate (F) values.

Experimental Setup

LLM-based experiments were conducted using two versions of the dataset. The initial dataset contained approximately 400 sentence–G-code pairs, while the refined dataset applied stricter annotation rules, explicit feed-rate normalization, and clearer intent definitions.

For the initial data set, the split of 75% training, 12.5% development, and 12.5% test samples was used. For the refined dataset, a 75% training and 25% test split was used.

The evaluation focused on three metrics: (i) exact match of the G-code, requiring full equality of the string with the reference command; (ii) precision of intent, defined as correct identification of the type of G-code command (G00, G01, or G28); and (iii) validity, measuring compliance with all safety constraints.

Fine-Tuned FLAN-T5 Base on the Initial Dataset

FLAN-T5 Base was fine-tuned on the initial dataset using a sequence-to-sequence learning strategy. The model was trained using the Adafactor optimizer with a learning rate of 2×10^{-5} for about ten epochs. Input sentences and target G-code commands were normalized for case and spacing.

The model was evaluated on both the development and test datasets. The performance of the fine-tuned FLAN-T5 Base model on the initial dataset is summarized in Table 6.4.

On the development dataset (38 samples), the model achieved an exact G-code match accuracy of 39.5On the test dataset (38 samples), the model achieved an exact-match accuracy of 23.7Although the exact-match accuracy was relatively low, all generated commands satisfied the predefined operational safety constraints, leading to a validity rate of 100

These results suggest that although the fine-tuned model was able to successfully identify the semantic intent of most commands, it was not able to accurately generate numerically

correct G-code commands, especially regarding the feed rate value. This problem led to further refinement of the dataset and the implementation of structured normalization rules in the next experiments.

Dataset	Exact Match (%)	Intent Accuracy (%)	Validity (%)
Development	39.5	81.6	100
Test	23.7	73.7	100

Table 6.4: Performance of the fine-tuned FLAN-T5 Base model on the initial dataset.

Fine-Tuned FLAN-T5 Base on the Refined Dataset

On the refined test dataset of 121 samples, the fine-tuned FLAN-T5 Base model showed a significant improvement over the experiments conducted on the initial dataset. The performance of the model on the refined test dataset is summarized in Table 6.5.

The model showed an intent accuracy of 95.0% and an entity accuracy of 90.9%, thus proving its excellent capability of accurately understanding operator instructions and identifying corresponding parameters. The accuracy of exact G-code matching and joint accuracy both improved to 71.1%, thus proving a significant improvement in the end-to-end executable command generation capability compared to the initial dataset experiments.

All the generated commands adhered to the predefined operational safety constraints, thus resulting in a validity rate of 100The above results prove that refining the dataset and imposing a strict feed-rate mapping constraint improved the numerical accuracy and reliability of the generated G-code commands.

A graphical representation of the component-wise and joint accuracies of the fine-tuned model is shown in Figure 6.4.

Metric	Value (%)
Number of samples (n)	121
Intent accuracy	95.0
Entity accuracy	90.9
G-code accuracy	71.1
Joint accuracy	71.1

Table 6.5: Performance of the fine-tuned FLAN-T5 Base model on the refined test set.

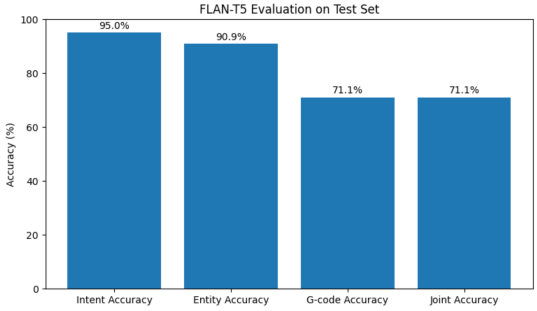

Figure 6.4: Component-wise and joint accuracies of the fine-tuned FLAN-T5 Base model on the refined test set.

Prompt-Based Inference with LLaMA-3.1 70B

The inference was carried out using LLaMA-3.1 70B through the Groq API. The model was tasked with the extraction of the intent and numerical parameters from natural language commands and the generation of a valid G-code instruction in a structured format.

Table 6.6 highlights the intent classification accuracy of the prompting-based inference on the refined dataset.

The model reported an average intent classification accuracy of about 66%, which is much lower than the accuracy reported by the Rasa-based system and the fine-tuned FLAN-T5 model. Class-wise analysis reported high precision for the *grind_linear* and *homing* classes, which indicates that the model was able to correctly classify these operations when predicted. However, the recall values were much lower, especially for the *grind_linear* class, which indicates that the model was prone to misclassifying grinding operations as pure positioning commands.

This is expected due to the limitation of the prompt-based inference approach, which is not fine-tuned for the specific task and hence reports lower consistency in intent classification and structured parameter extraction. Despite the classification inaccuracies, all the generated outputs satisfied the predefined safety constraints, reporting a validity rate of 100%.

Intent	Precision	Recall	F1-score	Support
grind_linear	0.92	0.56	0.69	79
homing	0.93	0.88	0.90	16
pure_positioning	0.40	0.83	0.54	30
Overall Accuracy	0.66 (125 samples)			

Table 6.6: Intent classification performance of LLaMA-3.1 70B using prompt-based inference on the refined dataset.

Prompt-Based Inference with LLaMA-3.1 70B (Prompt 2)

Another prompt configuration was tested on the same LLaMA-3.1 70B model and improved dataset. The goal of this experiment was to test the robustness of intent recognition and G-code generation against variations in prompt construction. The dataset, safety constraints, and testing procedure were maintained the same as in the first experiment involving prompts. Table 6.7 summarizes the component-wise and joint accuracies achieved using the revised prompt configuration.

The accuracy of intent classification was high at 94.5%, which shows that the refinement of the prompt is a significant improvement over the previous prompt configuration in terms of semantic understanding. The accuracy of entities was low at 49.0%, which shows that there are still challenges in accurately extracting the numerical parameters. The accuracy of G-code was 67.5%, and the accuracy of joints was only 44.5%.

These results demonstrate that prompt engineering alone can substantially improve intent recognition, but remains insufficient to guarantee reliable structured parameter extraction and exact executable command generation.

Metric	Value (%)
Number of samples (n)	200
Intent accuracy	94.5
Entity accuracy	49.0
G-code accuracy	67.5
Joint accuracy	44.5

Table 6.7: Performance of LLaMA-3.1 70B using prompt-based inference with the revised prompt configuration.

Figure 6.5 shows the distribution of true and predicted intent classes obtained using the revised prompt configuration. The figure highlights that most misclassifications occur between grinding and pure positioning commands, while homing commands are consistently recognized.

Figure 6.6 presents the component-wise and joint accuracy results achieved using prompt-based inference. The results confirm that although prompt refinement significantly improves intent classification performance, entity extraction and exact G-code generation remain the primary sources of error.

The figure highlights that most misclassifications occur between grinding and pure positioning commands, while homing commands are consistently recognized.

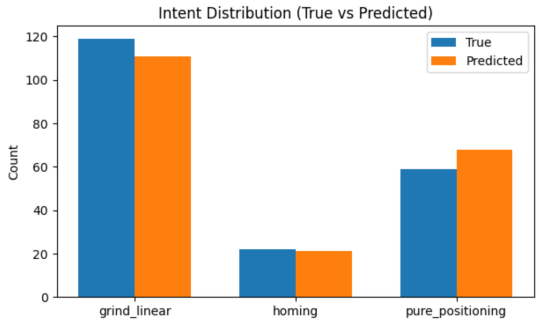

Figure 6.5: Distribution of true and predicted intent classes for prompt-based LLaMA-3.1 70B on the test set.

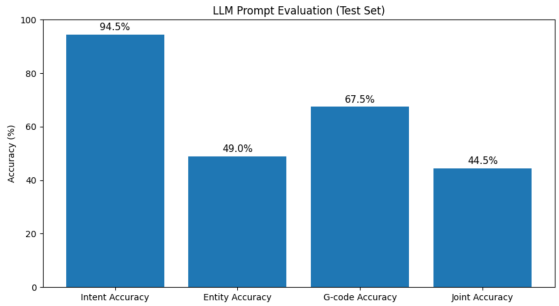

Figure 6.6: Component-wise and joint accuracies of prompt-based LLaMA-3.1 70B using the revised prompt configuration.

Pipeline-Based Inference Using LangChain

A pipeline-based approach using LangChain was evaluated on a subset of 100 samples. This approach decomposes the task into distinct stages, including intent classification, entity extraction, and G-code generation, allowing structured processing and intermediate validation.

Table 6.8 summarizes the component-wise and joint accuracies achieved by the LangChain-based pipeline.

The pipeline achieved an intent accuracy of 92%, confirming its ability to reliably identify the intended CNC operation. Entity accuracy reached 77%, indicating improved consistency in numerical parameter extraction compared to prompt-only LLM inference. G-code exact match accuracy and joint accuracy both reached 73%, demonstrating strong end-to-end performance in generating executable CNC commands.

These results show that the structured pipeline architecture significantly improves overall reliability by decomposing the task into modular stages, reducing error propagation and improving numerical consistency compared to purely prompt-based approaches.

Metric	Value (%)
Number of samples (n)	100
Intent accuracy	92
Entity accuracy	77
G-code exact match accuracy	73
Joint accuracy	73

Table 6.8: Performance of the LangChain-based LLM pipeline.

Figure 6.7 illustrates the component-wise and joint accuracies achieved by the LangChain-based pipeline.

The figure confirms that while intent classification performance is comparable to other approaches, the structured pipeline significantly improves entity extraction and overall joint accuracy compared to prompt-only inference.

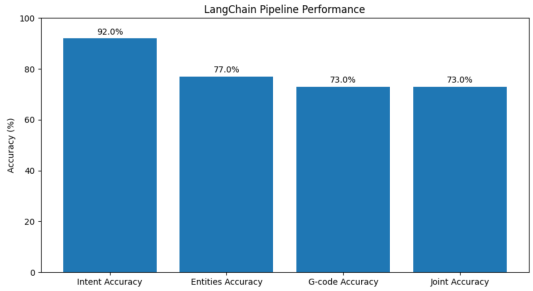

Figure 6.7: Component-wise and joint accuracies of the LangChain-based LLM pipeline on the test set.

Comparison of All Approaches

Figure 6.8 presents the joint accuracy comparison of the Rasa-based pipeline, the fine-tuned FLAN-T5 model, the LLaMA-3.1 70B model using Groq prompting, and the LangChain-based structured pipeline. The Rasa-based method had the highest joint accuracy of 98.8%, indicating the robustness of its deterministic design and rule-based feed rate inference. The LangChain pipeline had a joint accuracy of 73.0%, which shows that structured generation constraints have a substantial impact on execution accuracy. The fine-tuned FLAN-T5 model had a joint accuracy of 71.1%, indicating that supervised fine-tuning has improved structured output generation but is still facing challenges in numerical accuracy. On the other hand, the prompting-based LLaMA-3.1 70B model had a joint accuracy of 44.5%, which shows that prompting by itself is not enough to ensure the generation of exactly executable commands.

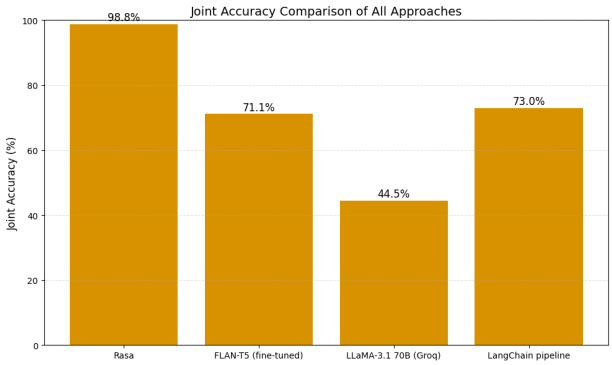

Figure 6.8: Joint accuracy comparison of all evaluated approaches on the test set

In addition to joint accuracy, a comprehensive comparison of component-wise performance for all approaches is illustrated in Figure 6.9.

This figure plots the intent accuracy, the extraction accuracy of the entity, the accuracy of the exact match of the G-code, and the joint accuracy for all approaches considered in this study.

The findings validate that all approaches have high intent classification accuracy, signifying robust semantic understanding of operator commands. However, a large performance gap is observed in entity extraction and G-code generation. The Rasa-based system consistently shows high performance on all fronts, signifying the strength of its structured intent and entity representation. The fine-tuned FLAN-T5 and LangChain-based systems show high intent classification accuracy but lower accuracy in entity extraction and exact G-code generation because of numerical errors. The prompt-based LLaMA-3.1 70B model shows high intent classification accuracy but lower entity and joint accuracy, signifying the limitations of prompt-based inference for structured industrial control tasks.

The findings of this study thus reiterate the significance of structured processing and parameter handling for robust end-to-end translation of natural language commands to executable CNC code.

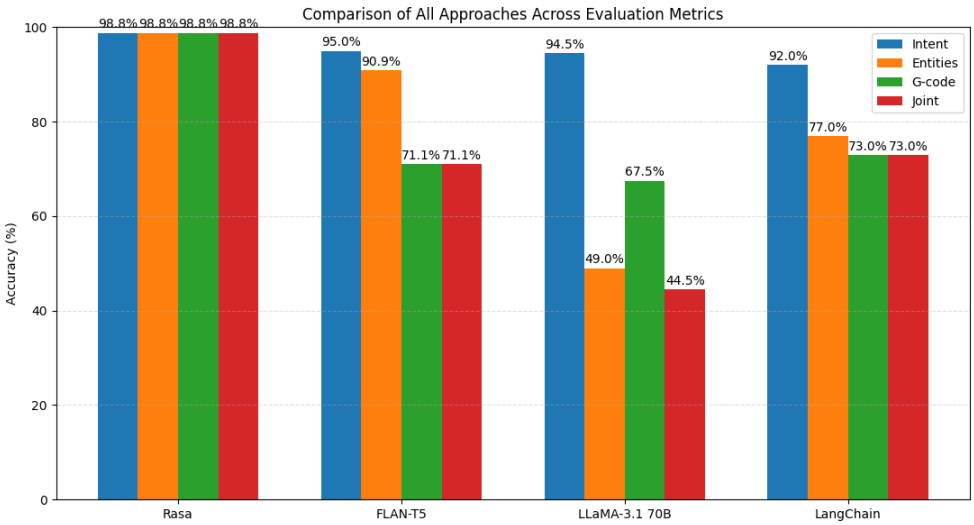

Figure 6.9: Component-wise and joint accuracy comparison across all evaluated approaches. Each group corresponds to one approach and includes intent, entity, G-code, and joint accuracy

Chapter 7

Discussion and Conclusions

The experimental findings of this thesis reveal a fundamental trade-off between deterministic structured NLU pipelines and large language model-based approaches when applied to safety-critical industrial automation tasks. Although both paradigms demonstrated strong capabilities in semantic interpretation of operator commands, their behavior diverges significantly when evaluated under strict numerical precision and safety constraints.

The Rasa-based system consistently achieved the highest joint accuracy and exact G-code correctness. This outcome is not accidental but rather a direct consequence of architectural design. The system decomposes the task into explicit subtasks: intent classification, entity extraction, rule-based feed-rate handling, parameter validation, and deterministic synthesis. Each stage is independently interpretable and bounded by explicit constraints. Such modular decomposition reduces uncertainty propagation. If one component fails, the failure mode is transparent and controllable.

In contrast, large language models approach the task holistically. They jointly infer semantic intent, parameter extraction, and output formatting in a single generative step. While this holistic modeling enables flexibility and robustness to linguistic variability, it also introduces instability in numeric reproduction. Generative models are inherently probabilistic; they optimize token likelihood rather than numeric exactness. Therefore, even when semantic intent is correctly understood, small deviations in feed-rate values frequently occur.

This discrepancy between high intent accuracy and lower exact G-code accuracy exposes a central challenge in natural language-to-code translation. Semantic understanding does not guarantee numeric fidelity. In industrial CNC control, numeric fidelity is not merely a formatting detail; it directly influences machine behavior and safety.

The dataset refinement process further clarifies this relationship. The introduction of stricter annotation rules, explicit feed-rate normalization policies, and approximately 100 additional

examples containing explicit F-numbers significantly improved performance, particularly for the fine-tuned FLAN-T5 model. This demonstrates that large language models are highly sensitive to annotation consistency and benefit from reduced ambiguity. However, it also reveals a dependency: generative systems require tightly controlled data distributions to achieve reliable numeric behavior.

The concept of functional correctness emerged as a necessary complement to exact string matching. Exact-match evaluation penalizes minor numeric variations even when those variations fall within operationally valid ranges. From an industrial perspective, a feed-rate difference between F10 and F12 within the same cutting category may not compromise machining safety. Therefore, evaluating only exact string equality may underestimate practical utility. Functional correctness, defined as compliance with motion type, axis values, and safety-constrained feed-rate ranges, provides a more realistic industrial metric.

Prompt-based inference experiments further illustrate the fragility of generative approaches. Minor changes in prompt wording caused substantial fluctuations in intent accuracy and feed-rate assignment. This sensitivity indicates that prompt engineering alone cannot guarantee stability in safety-critical environments. Although prompt refinement improved performance dramatically in certain cases, it remains an inherently heuristic process lacking formal guarantees.

In safety-critical industrial systems, deterministic behavior and predictable failure modes are essential. The structured Rasa-based pipeline offers clear advantages in this regard. Each decision is traceable, each parameter validated, and each failure explainable. This property makes structured NLU pipelines immediately more suitable for direct industrial deployment.

However, the results should not be interpreted as a rejection of large language models. On the contrary, LLMs exhibit superior flexibility in handling linguistic variation and capturing implicit semantic relationships. The challenge lies not in semantic comprehension but in numeric precision and constraint enforcement. Therefore, the future of natural language CNC control likely lies in hybrid architectures that combine semantic generalization capabilities of LLMs with deterministic validation layers.

7.1 Answers to the Research Questions

The findings of this thesis allow the research questions introduced in the literature review to be revisited in light of the experimental results.

With respect to the first research question, concerning the relationship between CNC programming and intelligent manufacturing, the present work confirms that the integration of language-based interfaces into CNC workflows is feasible, but only when combined with

strong operational constraints. Existing CNC research has largely focused on monitoring, optimization, and process analytics. The present results extend this line of work by showing that natural language can also function as an operator-facing interface layer, provided that the generated commands are validated before execution.

Regarding the second research question, related to the use of natural language interfaces in technical systems, the experiments confirm that techniques from task-oriented dialogue and machine command interpretation can be adapted to CNC applications. However, the results also show that CNC programming introduces stricter numeric and safety requirements than most conversational or robotic language interface settings.

The third research question addressed intent detection and semantic parsing methods for translating natural language into structured commands. The results strongly support the relevance of this paradigm. The Rasa-based structured pipeline consistently achieved the highest joint accuracy and the most reliable exact G-code generation, indicating that explicit decomposition into intent classification, entity extraction, and deterministic synthesis remains highly effective in safety-critical industrial contexts.

Finally, with respect to the fourth research question, concerning the application of large language models to structured code generation and CNC-related tasks, the results indicate that LLMs provide strong semantic flexibility but remain limited by numeric instability and prompt sensitivity. While fine-tuning and structured orchestration improve performance, purely generative approaches still require explicit validation and constraint enforcement to achieve industrial-grade reliability.

7.2 Practical and Theoretical Implications

From a practical perspective, the results suggest that natural language interfaces for CNC systems are most suitable as assistive layers rather than fully autonomous replacements for existing industrial programming workflows. In particular, structured NLU pipelines appear well suited for environments in which interpretability, traceability, and predictable failure modes are essential. Such systems could be deployed to support operator training, rapid command prototyping, or machine setup assistance while maintaining strict deterministic validation.

The findings also have implications for industrial adoption of large language models. Although LLMs exhibit strong flexibility in handling diverse linguistic formulations, their outputs cannot yet be assumed to satisfy the numeric and structural precision required in controller-level CNC programming. As a result, their most realistic short-term role is likely to be within hybrid architectures, where semantic interpretation is delegated to the language model while final command generation and validation remain under deterministic

control.

From a theoretical perspective, this thesis highlights an important distinction between semantic understanding and execution reliability. A model may correctly infer the intended machining action while still failing to reproduce the exact numeric parameters required for safe operation. This distinction is especially important in industrial NLP applications, where the success of a system cannot be measured solely by linguistic plausibility or even by high intent accuracy.

The results therefore support the view that future research on language interfaces for physical systems should treat constraint enforcement as a first-class design principle rather than as a post-processing convenience. In this sense, the thesis contributes not only to CNC-related automation research, but also to the broader discussion on how probabilistic language models can be integrated into safety-critical systems.

7.3 Limitations

This study is limited in several methodological and practical aspects. First, the system was evaluated on a single surface grinding machine with a restricted subset of CNC commands. Only linear grinding (G01), rapid positioning (G00), and homing (G28) were considered. Real industrial CNC systems support a far broader range of commands, including circular interpolation, dwell operations, tool changes, and multi-axis coordination. The restriction simplifies the problem and may limit generalizability.

Second, although the dataset consists of 500 sentenceG-code pairs, this scale remains modest relative to typical datasets used for training large language models. While sufficient for comparative experimentation, the dataset size constrains the statistical power of fine-tuning and limits generalization to unseen linguistic patterns. Future systems deployed in industrial settings would require significantly larger and more diverse corpora.

Third, feed-rate selection remains partially simplified. Although qualitative descriptors were mapped to validated numeric ranges, real machining operations depend on additional contextual variables such as material hardness, wheel wear, cooling conditions, and real-time sensor feedback. The current implementation does not incorporate dynamic process adaptation.

Another limitation concerns evaluation methodology. All experiments were conducted offline on held-out datasets. Real-time deployment would introduce additional constraints, including latency, robustness to speech recognition errors (if voice input is used), user behavior variability, and integration with industrial controllers. These practical aspects were not addressed in the present study.

The validation module enforces static parameter ranges. While these limits ensure safety within predefined boundaries, they do not account for adaptive machine states or evolving operational conditions.

An additional limitation concerns multilingual evaluation. Although the dataset includes both English and Italian operator commands, systematic experimental evaluation was conducted primarily on the English subset for the LLM-based approaches. The structured Rasa pipeline was configured to accept bilingual input and showed correct functional behavior on selected Italian examples. However, no large-scale quantitative comparison was performed to measure performance differences between English and Italian across all architectures. Consequently, the cross-lingual generalizability of the proposed approaches remains only partially validated.

7.4 Future Work

Several directions for future research arise from the findings of this thesis.

A primary direction involves the development of hybrid architectures. In such systems, a large language model would perform high-level semantic interpretation, while a deterministic rule-based engine would enforce numeric constraints and generate final executable commands. This separation could preserve semantic flexibility while guaranteeing safety.

Another promising avenue is constrained decoding. Instead of generating unrestricted text, language models could be forced to generate output under grammar constraints or symbolic templates. Techniques such as grammar-based decoding or token masking could significantly improve numeric precision.

Reinforcement learning or feedback-driven fine-tuning could also be explored. By penalizing numeric deviations during training, models might learn to prioritize exact numeric reproduction over stylistic variability.

Dataset expansion represents another critical direction. Larger and more diverse corpora, including multilingual operator commands and varied machining contexts, would enhance robustness. Incorporating Italian and English examples was a first step, but industrial systems may require broader linguistic coverage.

Future systems could also integrate real-time sensor feedback to dynamically adapt feed-rate values or validate commands against live machine state. This would move the system beyond static command translation toward intelligent machining assistance.

A further important direction concerns systematic multilingual evaluation. Although preliminary bilingual support was implemented for English and Italian in the structured pipeline, and the dataset includes Italian examples, comprehensive cross-lingual experiments

were not conducted. Future work should perform controlled comparative evaluations across multiple languages, including Italian and Persian, to assess portability of intent classification, numeric extraction, and constraint enforcement mechanisms. Such experiments would allow conclusions to be drawn regarding whether the proposed architectures generalize naturally across languages or require language-specific adaptation layers. A multilingual evaluation would also clarify the extent to which deterministic validation modules can remain language-independent while semantic interpretation components vary.

Finally, human-in-the-loop systems may provide a practical compromise between automation and safety. In such systems, AI-generated G-code would be reviewed or confirmed by an operator before execution, balancing efficiency gains with industrial risk management.

In conclusion, this thesis demonstrates that structured NLU pipelines currently provide superior reliability for safety-critical CNC command translation tasks. Large language models exhibit strong semantic capabilities but require additional constraint enforcement to achieve industrial-grade safety. The findings emphasize that in physical automation systems, semantic intelligence must be complemented by deterministic control and rigorous validation mechanisms.

Appendix A

Appendix: Selected Code Fragments

A.1 Example Rasa Outputs

```
{
  "intent": "grind_linear",
  "entities": {
    "x_value": 20,
    "z_value": -5,
    "F_label": "F_medium"
  },
  "gcode": "G01 X20 Z-5 F16"
}
```

A.2 Example Rasa Chatbot Interaction

```
User: Remove material up to point X20 Z-5 with a light cut.

System:
Intent: grind_linear
Entities: {x_value: 20, z_value: -5, F_label: F_medium}
System: Which exact F value do you want between 15 and 20?

User: 16

System:
```

```
Intent: grind_linear
Entities: {x_value: 20, z_value: -5, F_label: F_medium, f_value: 16}
Generated G-code: G01 X20 Z-5 F16

User: Move the wheel to X11 Z-4 at F8

System:
Intent: grind_linear
Entities: {x_value: 11, z_value: -4, f_value: 8}
Generated G-code: G01 X11 Z-4 F8
```

A.3 Example Outputs of the Rasa-Based System

```
Input: Advance the grinding wheel to X-10 Z-6.
Predicted intent: grind_linear
Extracted entities: X=-10, Z=-6
Generated G-code: G01 X-10 Z-6 F_standard

Input: Shift the grinding wheel to X0 Z-8 for a finishing pass.
Predicted intent: grind_linear
Extracted entities: X=0, Z=-8, cut_type=finishing
Generated G-code: G01 X0 Z-8 F_precise
```

A.4 Prompt Template for LLaMA-3.1 70B

```
You are a deterministic CNC compiler.

OUTPUT RULES (ABSOLUTE):
- Output MUST be valid JSON only (no extra text).
- Use DOUBLE QUOTES for all keys/strings.
- No markdown, no comments, no trailing commas.
- NEVER invent values.
- All numeric fields (x_value, z_value, F_number) MUST be JSON numbers
  (not strings).
- CONSISTENCY: numbers in x_value/z_value/F_number MUST exactly match
  the numbers used in gcode.

Return EXACTLY this schema:
```

```
    {"intent": "grind_linear | pure_positioning | homing",
    "x_value": number or null,
    "z_value": number or null,
    "descriptor": "precise | fine cut | light cut | rough cut | null",
    "F_number": number or null,
    "F_label": "F_precise | F_slow | F_medium | F_fast | F_standard
    | null",
    "gcode": "string"}

INTENT RULES (ABSOLUTE PRIORITY ORDER):
1) If ANY explicit F-number exists → intent = grind_linear
2) If cutting/grinding/removing material/spark-out/finishing/machining
  → intent = grind_linear
3) If home/origin/zero/return home → intent = homing
4) Else → intent = pure_positioning

DESCRIPTORS (normalize to one):
- precise   (spark-out, finishing pass)
- fine cut   (fine, slowly, carefully)
- light cut   (light, superficial)
- rough cut   (deep, heavy)
If none → null

DESCRIPTOR → F_label:
precise   → F_precise
fine cut   → F_slow
light cut   → F_medium
rough cut   → F_fast

F RULES:
- If explicit F-number exists → F_number = number and F_label = null
- Else if intent = grind_linear:
  - if descriptor exists → F_label from mapping
  - else → F_label = F_standard
- Else → F_number = null and F_label = null
```

A.5 Rasa-Based Pipeline Implementation

```
Input:
Posiziona la mola a X5.
```

```
Output:
{
  ”intent”: ”pure_positioning”,
  ”x_value”: 5,
  ”gcode”: ”G00 X5”
}
```

Bibliography

[ALE25] Mohamed Abdelaal, Samuel Lokadjaja, and Gilbert Engert. Gllm: Self-corrective g-code generation using large language models with user feedback. *arXiv preprint arXiv:2501.17584*, 2025.

[BMR+20] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[Bra14] Tim Bray. The javascript object notation (json) data interchange format. Technical report, IETF, 2014.

[CHL+24] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70):1–53, 2024.

[CKHT25] Khushi Choudki, Prarthana Kulkarni, Rutvik Hegde, and Varenya Thaker. Prompt-to-g-code generation using large language models for additive manufacturing. In *2025 9th International Conference on Computational System and Information Technology for Sustainable Solutions (CSITSS)*, pages 1–6. IEEE, 2025.

[CZZ+19] Jihong Chen, Kai Zhang, Yuan Zhou, Yufei Liu, Lingfeng Li, Zheng Chen, and Li Yin. Exploring the development of research, technology and business of machine tool domain in new-generation information technology environment based on machine learning. *Sustainability*, 11(12):3316, 2019.

[GGH+18] Chih-Wen Goo, Guang Gao, Yun-Kai Hsu, Chih-Li Huo, Tsung-Chieh Chen, Keng-Wei Hsu, and Yun-Nung Chen. Slot-gated modeling for joint slot filling and intent prediction. In *Proceedings of NAACL-HLT 2018 (Short Papers)*, pages 753–757, 2018.

[HBD+19] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*, 2019.

[JMG+24] Anushrut Jignasu, Kelly Marshall, Baskar Ganapathysubramanian, Aditya Balu, Chinmay Hegde, and Adarsh Krishnamurthy. Evaluating large language models for g-code debugging, manipulation, and comprehension. In *2024 IEEE LLM Aided Design Workshop (LAD)*, pages 1–5. IEEE, 2024.

[KD18] Aishwarya Kamath and Rajarshi Das. A survey on semantic parsing. *arXiv preprint arXiv:1812.00978*, 2018.

[KS24] S Kanimozhi and YS Sriker. Explorative deployment of fine-tuned large language model for on-site computerized numeric control machine operator assistance. In *2024 IEEE Silchar Subsection Conference (SILCON)*, pages 1–6. IEEE, 2024.

[LAYK21] Kamran Latif, Anbia Adam, Yusri Yusof, and Aini Zuhra Abdul Kadir. A review of g code, step, step-nc, and open architecture control technologies based embedded cnc systems. *The International Journal of Advanced Manufacturing Technology*, 114(9):2549–2566, 2021.

[LVHK24] Jonghan Lim, Birgit Vogel-Heuser, and Ilya Kovalenko. Large language model-enabled multi-agent manufacturing systems. In *2024 IEEE 20th International Conference on Automation Science and Engineering (CASE)*, pages 3940–3946. IEEE, 2024.

[LZW+24] Diya Li, Yue Zhao, Zhifang Wang, Calvin Jung, and Zhe Zhang. Large language model-driven structured output: A comprehensive benchmark and spatial data generation framework. *ISPRS International Journal of Geo-Information*, 13(11):405, 2024.

[MASA22] Divyansh Shankar Mishra, Abhinav Agarwal, BP Swathi, and K C Akshay. Natural language query formalization to sparql for querying knowledge bases using rasa. *Progress in Artificial Intelligence*, 11(3):193–206, 2022.

[NPB20] Trung Kien Nguyen, Lan Xuan Phung, and Ngoc-Tam Bui. Novel integration of capp in a g-code generation module using macro programming for cnc application. *Machines*, 8(4):61, 2020.

[NRP+18] Daniel Nyga, Subhro Roy, Rohan Paul, Daehyung Park, Mihai Pomarlan, Michael Beetz, and Nicholas Roy. Grounding robot plans from natural language instructions with incomplete world knowledge. In *Conference on Robot Learning*, pages 714–723. PMLR, 2018.

[NZZ+23] Linyong Nan, Yilun Zhao, Weijin Zou, Narutatsu Ri, Jaesung Tae, Ellen Zhang, Arman Cohan, and Dragomir Radev. Enhancing text-to-sql capabilities of large language models: A study on prompt design strategies. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 14935–14956, 2023.

[PMN25] Yasaman Hashem Pour, Nazanin Mahjourian, and Vinh Nguyen. Few-shot vlm-based g-code and hmi verification in cnc machining. *arXiv preprint arXiv:2512.11296*, 2025.

[QLC+21] Libo Qin, Tailu Liu, Wanxiang Che, Bingbing Kang, Sendong Zhao, and Ting Liu. A co-interactive transformer for joint slot filling and intent detection. In *ICASSP 2021 IEEE International Conference on Acoustics, Speech and Signal Processing*, pages 8193–8197. IEEE, 2021.

[RLB22] Nitarshan Rajkumar, Raymond Li, and Dzmitry Bahdanau. Evaluating the text-to-sql capabilities of large language models. *arXiv preprint arXiv:2204.00498*, 2022.

[RS20] Diego Reforgiato Recupero and Federico Spiga. Knowledge acquisition from parsing natural language expressions for humanoid robot action commands. *Information Processing & Management*, 57(6):102094, 2020.

[SCB+24] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2024.

[SHP22] Barun Kumar Saha, Luca Haab, and Łukasz Podleski. Intent-based industrial network management using natural language instructions. In *2022 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT)*, pages 1–6. IEEE, 2022.

[SME+24] Anna-Maria Schmitt, Eddi Miller, Bastian Engelmann, Rafael Batres, and Jan Schmitt. G-code evaluation in cnc milling to predict energy consumption through machine learning. *Advances in Industrial and Manufacturing Engineering*, 8:100140, 2024.

[ŠPB+25] Kristijan Šket, David Potočnik, Miran Brezočnik, Mirko Ficko, and Simon Klančnik. Large language models for g-code generation in cnc machining: A comparison of chatgpt-3.5 and chatgpt-4o. *Advances in Production Engineering & Management*, 20(2):224–238, 2025.

[SSB21] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. Picard: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of EMNLP 2021*, pages 9895–9901, 2021.

[TLI+23] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[VSP+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[WC23] Jianxun Wang and Yixiang Chen. A review on code generation with llms: Application and evaluation. In *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)*, pages 284–289. IEEE, 2023.

[WDS+20] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, pages 38–45, 2020.

[WPC19] Nick Walker, Yu-Tang Peng, and Maya Cakmak. Neural semantic parsing with anonymization for command understanding in general-purpose service robots. In *Robot World Cup*, pages 337–350. Springer, 2019.

[WRK+21] Christopher Wang, Candace Ross, Yen-Ling Kuo, Boris Katz, and Andrei Barbu. Learning a natural-language to ltl executable semantic parser for grounded robotics. In *Proceedings of the 5th Conference on Robot Learning*, volume 155 of *Proceedings of Machine Learning Research*, pages 1706–1718. PMLR, 2021.

[WSM+25] Darren Yow-Bang Wang, Zhengyuan Shen, Soumya Smruti Mishra, Zhichao Xu, Yifei Teng, and Haibo Ding. Slot: Structuring the output of large language models. *arXiv preprint arXiv:2505.04016*, 2025.

[WWR+21] Jixuan Wang, Kai Wei, Martin Radfar, Weiwei Zhang, and Clement Chung. Encoding syntactic knowledge in transformer encoder for intent detection and slot filling. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 13943–13951, 2021.

[WWS+22] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

[YRST24] Ziyi Yang, Shreyas S Raman, Ankit Shah, and Stefanie Tellex. Plug in the safety chip: Enforcing constraints for llm-driven robot agents. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 14435–14442. IEEE, 2024.